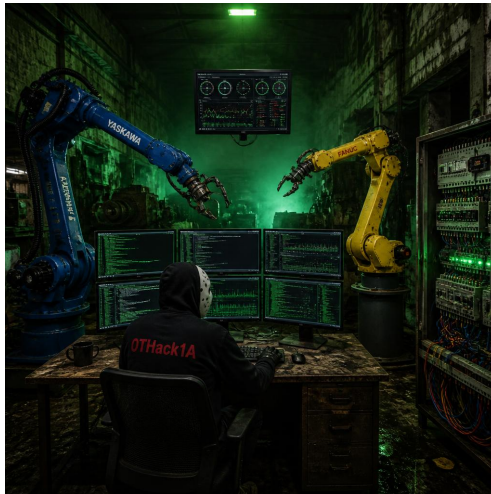




Il Kernel Linux

Architettura, Internals e Sviluppo

Ambiente: Debian 12 • Arch Linux • Kernel 6.x LTS

Codice C, esempi pratici verificati, analisi in profondità dei sottosistemi

Creator:

0THack1A

<https://github.com/0THack1A>

Documento personale — Solo uso didattico

Il Kernel Linux — Architettura e Internals

Creator: OTHack1A <https://github.com/OTHack1A>

Documento personale a uso esclusivamente didattico.

Linux[®] è un marchio registrato di Linus Torvalds.

Esempi testati su Debian 12 Bookworm (kernel 6.1-LTS) e Arch Linux (kernel 6.9/6.10).

Fonti principali:

The Linux Kernel documentation — <https://docs.kernel.org>

Linux Kernel Internals — <https://kernel-internals.org>

kernel.org git — <https://git.kernel.org>

Linux Weekly News — <https://lwn.net>

Arch Wiki — <https://wiki.archlinux.org>

Indice

Prefazione	5
1 Architettura del Kernel Linux	6
1.1 Kernel space vs User space	6
1.2 Sottosistemi principali	7
1.3 La System Call Interface	7
1.4 Struttura del sorgente kernel	8
1.5 Versioni e ciclo di sviluppo	9
2 Processi e Thread	11
2.1 task_struct: la struttura centrale	11
2.2 Ciclo di vita di un processo	12
2.3 Thread: clone() e POSIX threads	13
2.4 Segnali	14
2.5 Processi zombie e orfani	16
3 Lo Scheduler del Kernel	18
3.1 Classi di scheduling	18
3.2 CFS: Completely Fair Scheduler	19
3.3 EEVDF: il nuovo algoritmo dal kernel 6.6	20
3.4 sched_ext: scheduler programmabili con eBPF	20
3.5 Real-time scheduling	22
4 Gestione della Memoria	24
4.1 Memoria fisica e pagine	24
4.2 Buddy System e allocazione pagine	25
4.3 Slab Allocator: SLUB	25
4.4 Memoria virtuale e VMA	27
4.5 Page fault e Copy-on-Write	28
4.6 OOM Killer	29
5 VFS: Virtual Filesystem Switch	30
5.1 Strutture dati fondamentali	30
5.2 File operations: l'interfaccia driver-VFS	31
5.3 Page cache e writeback	32
5.4 Filesystem principali	33
5.5 /proc e /sys: filesystem virtuali	34
6 Driver e Moduli Kernel	35
6.1 Anatomia di un modulo kernel	35
6.2 Modulo con parametri	36
6.3 Character device driver	37
6.4 Dipendenze tra moduli e blacklist	39

7	Sottosistema di Rete	41
7.1	Architettura dello stack di rete	41
7.2	Socket e comunicazione di rete	42
7.3	Netfilter e nftables	43
7.4	XDP: eXpress Data Path	44
7.5	Diagnostica di rete avanzata	45
8	Interrupt e Gestione Hardware	46
8.1	Tipi di interrupt	46
8.2	Top half e bottom half	47
8.3	Sincronizzazione in interrupt context	48
8.4	DMA: Direct Memory Access	49
9	Sincronizzazione e Locking	50
9.1	Primitive di sincronizzazione	50
9.2	Spinlock	50
9.3	Mutex	51
9.4	RCU: Read-Copy-Update	51
9.5	Operazioni atomiche	52
9.6	Lockdep: rilevamento deadlock	53
9.7	seqlock e read-mostly patterns	54
10	eBPF: Programmabilità del Kernel	55
10.1	Architettura eBPF	55
10.2	Primo programma eBPF	55
10.3	Mappe eBPF	57
10.4	bpftool: ispezione del sottosistema eBPF	58
10.5	CO-RE: portabilità tra kernel	59
11	Namespaces, Cgroups e Container	60
11.1	Namespace Linux	60
11.2	Cgroups v2	61
11.3	Container: come funziona Docker internamente	62
11.4	Seccomp: filtro system call	63
12	Sicurezza del Kernel: LSM e Hardening	65
12.1	Linux Security Modules (LSM)	65
12.2	Capabilities Linux	66
12.3	Hardening del kernel	67
12.4	AppArmor: profili per il kernel	68
A	Strumenti di Analisi e Debug	70
B	Fonti e Riferimenti	72
B.1	Documentazione ufficiale	72
B.2	Libri di riferimento	72
B.3	Sorgenti online	72

Glossario**73**

Elenco delle figure

1.1	Strati dell'architettura kernel Linux	7
-----	---	---

Elenco delle tabelle

1.1	Protection rings in Linux	6
1.2	Sottosistemi principali del kernel Linux	7
1.3	Rami del kernel Linux e loro ciclo	9
2.1	Stati di un processo Linux	13
2.2	Segnali principali e loro gestione predefinita	15
3.1	Classi di scheduling in ordine di priorit� decrescente	18
5.1	Filesystem Linux a confronto tecnico	33
7.1	Percorso di un pacchetto in ricezione	42
8.1	Tipi di interrupt in Linux	46
9.1	Primitive di sincronizzazione nel kernel Linux	50
10.1	Componenti del sottosistema eBPF	55
11.1	I sette tipi di namespace Linux	60
12.1	Capability Linux principali	66
A.1	Strumenti fondamentali per l'analisi del kernel	70

Prefazione

Questo documento è un approfondimento tecnico sull'architettura interna del kernel Linux. Non è pensato come introduzione all'uso di Linux, ma come guida per chi vuole capire cosa succede *sotto il cofano*: come vengono gestiti i processi, la memoria, i dispositivi, la rete, la sicurezza.

Il riferimento principale è il kernel **6.x LTS** (ramo 6.1, Long Term Support fino a dicembre 2026, SLTS fino al 2033), con note sui cambiamenti introdotti nelle versioni 6.6, 6.9 e 6.10.

Struttura del documento:

1. Architettura generale e spazio utente/kernel
2. Gestione dei processi e thread
3. Scheduler: CFS, EEVDF, sched_ext
4. Gestione della memoria virtuale
5. Filesystem VFS e implementazioni
6. Driver e moduli kernel
7. Sottosistema di rete
8. Interrupt e gestione hardware
9. Sincronizzazione e locking
10. eBPF: programmabilità del kernel
11. Namespaces, cgroups e container
12. Sicurezza: LSM, SELinux, AppArmor

OTHack1A

Capitolo 1

Architettura del Kernel Linux

Il kernel Linux è un **kernel monolitico modulare**: tutto il codice privilegiato gira in un unico spazio di indirizzamento (*kernel space*), ma le funzionalità possono essere caricate e scaricate dinamicamente come *moduli*.

1.1 Kernel space vs User space

La CPU moderna implementa almeno due livelli di privilegio (*protection rings*). Intel/AMD ne definisce quattro (ring 0–3); Linux ne usa due:

Tabella 1.1: Protection rings in Linux

Ring	Nome	Caratteristiche
Ring 0	Kernel space	Accesso diretto all'hardware, a tutta la RAM, alle istruzioni privilegiate (HLT, IN/OUT, CR0/CR3...)
Ring 3	User space	Memoria isolata, accesso hardware solo tramite system call, fault non bloccano il sistema

Il confine tra i due mondi è attraversato solo tramite le **system call** (syscall), le eccezioni hardware e gli interrupt.

Listing 1.1: Visualizzare il confine kernel/user in /proc

```
1 # Mappa di memoria di un processo (sezioni kernel in alto)
2 cat /proc/self/maps | tail -5
3
4 # Statistiche syscall del processo corrente
5 strace -c ls /tmp 2>&1 | head -20
6
7 # Numero totale di syscall definite nel kernel corrente
8 ausyscall --dump 2>/dev/null | wc -l
9
10 # Quali syscall usa un programma
11 strace -e trace=all cat /etc/hostname
```

1.2 Sottosistemi principali

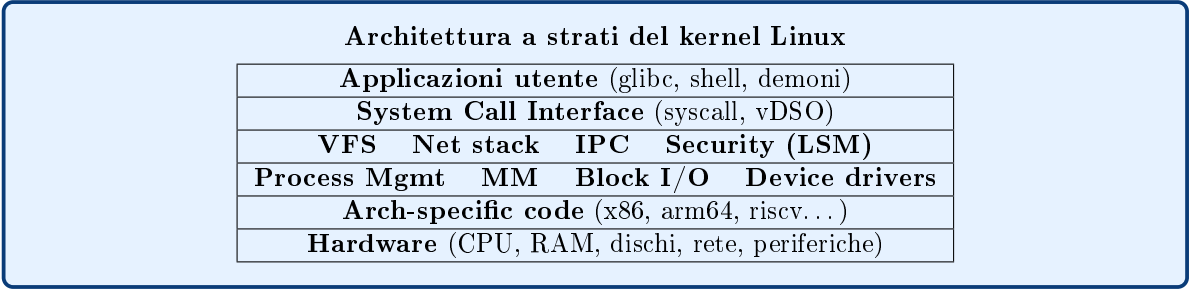


Figura 1.1: Strati dell’architettura kernel Linux

Tabella 1.2: Sottosistemi principali del kernel Linux

Sottosistema	Directory sorgente	Funzione principale
Process Management	kernel/	Fork, exec, segnali, scheduling, namespace PID
Memory Management	mm/	Pagine fisiche, VMAs, slab allocator, swap, huge pages
VFS	fs/	Astrazione filesystem, inode/dentry cache, buffer I/O
Network stack	net/	Socket, TCP/IP, netfilter, routing, XDP
Device drivers	drivers/	Interfaccia hardware: block, char, USB, PCI, GPU
Interrupt/IRQ	kernel/irq/	Gestione interrupt hardware e software (softirq, tasklet)
Security (LSM)	security/	Hook di sicurezza, SELinux, AppArmor, seccomp
eBPF	kernel/bpf/	Verifier, JIT, mappe, programmi in-kernel sicuri
Cgroups/NS	kernel/cgroup/	Isolamento e controllo risorse per container
Arch	arch/	Codice specifico per x86_64, arm64, riscv, mips...

1.3 La System Call Interface

Le system call sono il meccanismo fondamentale con cui il codice user space chiede servizi al kernel. Su x86_64 si invocano tramite l’istruzione `syscall`; il numero della syscall viene passato in `rax`, i parametri in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`.

Listing 1.2: Sistema call: dal C all’assembly (x86_64)

```
1 /* In glibc: write(fd, buf, count) diventa: */
2 /* mov rax, 1          ; SYS_write = 1          */
3 /* mov rdi, fd          */
4 /* mov rsi, buf          */
5 /* mov rdx, count       */
```

```

6  /* syscall */
7
8  /* Implementazione kernel: fs/read_write.c */
9  SYSCALL_DEFINE3(write, unsigned int, fd,
10                  const char __user *, buf,
11                  size_t, count)
12 {
13     struct fd f = fdget_pos(fd);
14     ssize_t ret = -EBADF;
15
16     if (f.file) {
17         loff_t pos, *ppos = file_ppos(f.file);
18         if (ppos) {
19             pos = *ppos;
20             ppos = &pos;
21         }
22         ret = vfs_write(f.file, buf, count, ppos);
23         if (ret >= 0 && ppos)
24             f.file->f_pos = pos;
25         fdput_pos(f);
26     }
27     return ret;
28 }

```

Listing 1.3: Esaminare le syscall del sistema

```

1  # Tabella delle syscall su x86_64
2  cat /usr/include/x86_64-linux-gnu/asm/unistd_64.h | \
3  grep "#define __NR_" | head -30
4
5  # Traccia le syscall di un processo in esecuzione
6  sudo strace -p $(pgrep sshd | head -1) -e trace=network 2>&1 &
7
8  # Conta le syscall per tipo (profiling)
9  sudo perf stat -e syscalls:sys_enter_read,\
10 syscalls:sys_enter_write ls /etc 2>&1
11
12 # vDSO: syscall senza context switch (clock_gettime, gettimeofday)
13 cat /proc/self/maps | grep vdso
14 # Output: 7ffd2b9fe000-7ffd2ba00000 r-xp 00000000 00:00 0 [vdso]

```

1.4 Struttura del sorgente kernel

Listing 1.4: Scaricare e navigare il sorgente kernel

```

1  # Scarica sorgente kernel (Debian)
2  sudo apt install linux-source
3  cd /usr/src && sudo tar xf linux-source-*.tar.xz
4  cd linux-source-*/
5
6  # Su Arch Linux
7  sudo pacman -S linux-headers
8  ls /usr/src/linux/
9
10 # Struttura top-level

```

```

11 ls linux-source-6.1/
12 # arch/      drivers/  fs/      include/  init/
13 # ipc/       kernel/   lib/     mm/       net/
14 # security/  sound/    tools/   usr/      virt/
15
16 # Cerca la definizione di task_struct
17 grep -r "struct task_struct {" \
18     linux-source-6.1/include/linux/ 2>/dev/null
19
20 # Cerca l'implementazione di fork
21 grep -n "SYSCALL_DEFINE" \
22     linux-source-6.1/kernel/fork.c | head -10
23
24 # Dimensione del kernel compilato
25 ls -lh /boot/vmlinuz-$(uname -r)
26
27 # Configurazione corrente
28 zcat /proc/config.gz 2>/dev/null | grep CONFIG_SMP
29 # oppure:
30 cat /boot/config-$(uname -r) | grep CONFIG_HZ

```

1.5 Versioni e ciclo di sviluppo

Tabella 1.3: Rami del kernel Linux e loro ciclo

Ramo	Esempio	Caratteristiche
mainline	6.10-rc3	Sviluppo attivo, Linus Torvalds, merge window
stable	6.9.x	Backport bugfix, 2-3 mesi di vita
LTS	6.1.x	Long Term Support, 2+ anni di aggiornamenti
SLTS	6.1.x	Super-LTS (CIP), fino al 2033
EOL	6.0.x	End of life, non piu' aggiornato
distro	6.1-debian	Patch specifiche della distribuzione

Listing 1.5: Verificare e aggiornare il kernel

```

1 # Versione corrente
2 uname -r
3 # 6.1.0-20-amd64
4
5 # Kernel installati (Debian)
6 dpkg --get-architecture | grep linux-image
7
8 # Installa kernel LTS su Debian
9 sudo apt install linux-image-6.1.0-20-amd64
10
11 # Su Arch: kernel mainline o LTS
12 sudo pacman -S linux          # mainline
13 sudo pacman -S linux-lts      # LTS
14
15 # Compilare il kernel dalla sorgente
16 cd /usr/src/linux-source-6.1/
17 make menuconfig              # interfaccia ncurses
18 make -j$(nproc)              # compila con tutti i core

```

```
19 sudo make modules_install
20 sudo make install
21
22 # Controlla la versione del kernel in esecuzione
23 cat /proc/version
24 # Linux version 6.1.0-20-amd64 (debian-kernel@lists.debian.org)
```

Kernel monolitico vs microkernel

Linux è monolitico: tutti i driver, filesystem e protocolli di rete girano in kernel space. I **microkernel** (GNU Mach, QNX, seL4) spostano questi componenti in processi user space separati, aumentando l'isolamento ma riducendo le performance. Linux compensa con la modularità dinamica (moduli `.ko`) e eBPF.

Capitolo 2

Processi e Thread

2.1 task_struct: la struttura centrale

Ogni processo (e thread) nel kernel è rappresentato da una `struct task_struct`, definita in `include/linux/sched.h`. È la struttura dati più importante del kernel: contiene tutto ciò che il kernel sa di un task.

Listing 2.1: Campi principali di `task_struct` (kernel 6.1, semplificato)

```
1  /* include/linux/sched.h */
2  struct task_struct {
3      /* Stato del processo */
4      unsigned int    __state;    /* TASK_RUNNING, TASK_INTERRUPTIBLE...
5                                   */
6      int             exit_state;
7      int             exit_code;
8
9      /* Identita' */
10     pid_t            pid;         /* Process ID */
11     pid_t            tgid;        /* Thread Group ID (= PID del leader)
12                                   */
13     struct task_struct *real_parent;
14     struct task_struct *parent;
15
16     /* Credenziali */
17     const struct cred *cred;      /* UID, GID, capabilities */
18
19     /* Scheduling */
20     int              prio;         /* priorita' dinamica */
21     int              static_prio;
22     int              normal_prio;
23     unsigned int     rt_priority;
24     struct sched_entity se;        /* entita' CFS */
25     const struct sched_class *sched_class;
26
27     /* Memoria */
28     struct mm_struct *mm;          /* mappa di memoria (NULL per kthread)
29                                   */
30     struct mm_struct *active_mm;
31
32     /* File system */
33     struct fs_struct *fs;          /* cwd, root */
34     struct files_struct *files;    /* tabella dei file descriptor aperti
35                                   */
36
37     /* Segnali */
38     struct signal_struct *signal;
39     struct sighand_struct *sighand;
```

```

36     sigset_t          blocked;  /* maschera segnali bloccati */
37
38     /* Thread info */
39     void              *stack;    /* kernel stack */
40     struct thread_info thread_info;
41
42     /* Namespace */
43     struct nsproxy     *nsproxy; /* PID, net, mnt, uts, ipc NS */
44
45     /* cgroups */
46     struct css_set     *cgroups;
47 };

```

Listing 2.2: Ispezionare task_struct da spazio utente

```

1  # PID e Tgid di un processo
2  cat /proc/self/status | grep -E "^(Pid|Tgid|PPid):"
3  # Pid: 12345
4  # Tgid: 12345
5  # PPid: 12344
6
7  # Stato del processo
8  cat /proc/self/status | grep "^State:"
9  # State: R (running)
10
11 # Stati possibili:
12 # R = Running (in esecuzione o pronto)
13 # S = Sleeping (interruptible)
14 # D = Disk sleep (uninterruptible, I/O)
15 # T = Stopped (SIGSTOP o traced)
16 # Z = Zombie (terminato, parent non ha fatto wait)
17 # I = Idle kernel thread
18
19 # Visualizza task_struct via /proc
20 cat /proc/1/status          # init/systemd
21 cat /proc/self/wchan        # funzione kernel in cui il proc. dorme
22
23 # Tutti i thread di un processo (ps mostra thread con -L)
24 ps -elf | grep firefox | head -5

```

2.2 Ciclo di vita di un processo

Listing 2.3: fork(), exec() e wait() — il ciclo fondamentale

```

1  /* Creare un processo figlio: fork() */
2  /* kernel/fork.c -> copy_process() */
3  pid_t fork(void)
4  {
5      /* 1. Alloca nuova task_struct */
6      /* 2. Copia mm_struct (copy-on-write) */
7      /* 3. Copia file descriptor table */
8      /* 4. Assegna nuovo PID */
9      /* 5. Aggiunge alla run queue */
10     return do_fork(SIGCHLD, 0, 0, NULL, NULL);
11 }

```

```

12
13  /* Esempio completo: processo padre/figlio */
14  #include <stdio.h>
15  #include <unistd.h>
16  #include <sys/wait.h>
17
18  int main(void) {
19      pid_t pid = fork();
20
21      if (pid < 0) {
22          perror("fork");
23          return 1;
24      } else if (pid == 0) {
25          /* Figlio: esegue un nuovo programma */
26          execl("/bin/ls", "ls", "-la", "/tmp", NULL);
27          perror("execl"); /* mai raggiunto se exec ha successo */
28          return 1;
29      } else {
30          /* Padre: attende il figlio */
31          int status;
32          waitpid(pid, &status, 0);
33          printf("Figlio %d uscito con %d\n",
34              pid, WEXITSTATUS(status));
35      }
36      return 0;
37  }

```

Tabella 2.1: Stati di un processo Linux

Stato	Costante kernel	Descrizione
Running	TASK_RUNNING	Sulla run queue, esegue o pronto
Sleeping	TASK_INTERRUPTIBLE	Attende evento, svegliabile da segnale
D-sleep	TASK_UNINTERRUPT	Attende I/O, non svegliabile
Stopped	TASK_STOPPED	Fermato da SIGSTOP/SIGTSTP
Traced	TASK_TRACED	Sotto ptrace (gdb, strace)
Zombie	EXIT_ZOMBIE	Terminato, parent non ha fatto wait()
Dead	EXIT_DEAD	In fase di de-allocazione finale

2.3 Thread: clone() e POSIX threads

In Linux, thread e processi sono implementati con la stessa struttura `task_struct`. La differenza sta nelle risorse **condivise**: i thread condividono lo spazio di indirizzamento, i file descriptor e il signal handler.

Listing 2.4: clone() — il syscall alla base di fork e pthread

```

1  /*
2   * clone() e' il syscall reale; fork() e' un wrapper.
3   * I flag determinano cosa viene condiviso:
4   *
5   * CLONE_VM      : condividi mm_struct (thread)
6   * CLONE_FS      : condividi fs_struct (cwd, root)
7   * CLONE_FILES   : condividi files_struct (fd table)
8   * CLONE_SIGHAND : condividi sighand_struct

```

```

9  * CLONE_THREAD : stesso thread group (TGID)
10 * CLONE_NEWPID : nuovo namespace PID (container)
11 * CLONE_NEWNET : nuovo namespace rete
12 */
13
14 /* fork() equivale a: */
15 clone(SIGCHLD, 0, NULL, NULL, NULL);
16
17 /* pthread_create() equivale a: */
18 clone(CLONE_VM | CLONE_FS | CLONE_FILES |
19       CLONE_SIGHAND | CLONE_THREAD |
20       CLONE_SETTLS | CLONE_PARENT_SETTID |
21       CLONE_CHILD_CLEAR_TID | SIGCHLD, ...);
22
23 /* Esempio: crea thread con pthreads */
24 #include <pthread.h>
25 #include <stdio.h>
26
27 void *thread_fn(void *arg) {
28     printf("Thread TID: %d, PID: %d\n",
29           getpid(), gettid()); /* stesso PID, diverso TID */
30     return NULL;
31 }
32
33 int main(void) {
34     pthread_t t;
35     pthread_create(&t, NULL, thread_fn, NULL);
36     pthread_join(t, NULL);
37     return 0;
38 }

```

Listing 2.5: Visualizzare processi e thread

```

1  # Albero processi con PID e PPID
2  pstree -p | head -20
3
4  # Lista thread (LWP = Light Weight Process)
5  ps -elf | grep -v grep | awk 'NR==1 || /bash/'
6  # UID  PID  PPID  LWP  ... CMD
7  # mario 100  99    100  ... bash    <- thread principale
8
9  # Thread di un processo specifico
10 ls /proc/$(pgrep -n firefox)/task/
11
12 # Info scheduler per processo
13 cat /proc/$(pgrep -n bash)/sched | head -15
14
15 # Numero di thread per processo (ordinati)
16 ps -eo pid,nlwp,comm --sort=-nlwp | head -10

```

2.4 Segnali

I segnali sono notifiche asincrone inviate a un processo. Il kernel gestisce i segnali tramite i campi `signal_struct` e `sighand_struct` di `task_struct`.

Tabella 2.2: Segnali principali e loro gestione predefinita

Num.	Nome	Default	Causa tipica
1	SIGHUP	Termina	Disconnessione terminale
2	SIGINT	Termina	Ctrl+C
3	SIGQUIT	Core dump	Ctrl+\
6	SIGABRT	Core dump	abort()
9	SIGKILL	Termina	Non catturabile
11	SIGSEGV	Core dump	Accesso memoria invalido
13	SIGPIPE	Termina	Scrittura su pipe senza lettori
14	SIGALRM	Termina	Timer scaduto (alarm())
15	SIGTERM	Termina	Terminazione soft (default kill)
17	SIGCHLD	Ignora	Figlio terminato
18	SIGCONT	Continua	Riprendi processo fermato
19	SIGSTOP	Ferma	Non catturabile
29	SIGIO	Ignora	I/O disponibile (aio)

Listing 2.6: Gestione segnali in C

```

1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 /* Handler personalizzato */
6 static volatile sig_atomic_t running = 1;
7
8 void handle_sigterm(int sig) {
9     (void)sig;
10     running = 0;    /* operazione atomica: sicura nell'handler */
11 }
12
13 int main(void) {
14     struct sigaction sa = {
15         .sa_handler = handle_sigterm,
16         .sa_flags    = SA_RESTART,    /* riavvia syscall interrotte */
17     };
18     sigemptyset(&sa.sa_mask);
19     sigaction(SIGTERM, &sa, NULL);
20     sigaction(SIGINT, &sa, NULL);
21
22     /* Blocca SIGUSR1 temporaneamente */
23     sigset_t mask;
24     sigemptyset(&mask);
25     sigaddset(&mask, SIGUSR1);
26     sigprocmask(SIG_BLOCK, &mask, NULL);
27
28     printf("PID %d in esecuzione...\n", getpid());
29     while (running) {
30         pause(); /* attende segnale */
31     }
32     printf("Uscita pulita\n");
33     return 0;
34 }

```

Listing 2.7: Gestione segnali da shell

```

1  # Invia segnali
2  kill -SIGTERM 1234
3  kill -9 1234          # SIGKILL, non catturabile
4  kill -0 1234          # verifica esistenza processo (no invio reale)
5  killall -SIGHUP nginx
6  pkill -USR1 logrotate
7
8  # Maschera segnali di un processo
9  cat /proc/self/status | grep Sig
10 # SigBlk: 0000000000000000 (maschera bloccati, hex)
11 # SigIgn: 0000000000001000 (maschera ignorati)
12 # SigCgt: 0000000180014202 (handler installati)
13
14 # Decodifica maschera segnali
15 python3 -c "
16 mask = 0x0000000180014202
17 for i in range(1, 65):
18     if mask & (1 << (i-1)):
19         print(f'Segnale {i} ha handler')
20 "

```

2.5 Processi zombie e orfani

Listing 2.8: Zombie e orfani — gestione con wait()

```

1  /*
2   * ZOMBIE: un processo figlio che ha terminato
3   * ma il cui padre non ha ancora chiamato wait().
4   * Occupa una entry in process table ma non memoria.
5   * Si vede come 'Z' in ps.
6   */
7
8  /* Prevenire zombie: wait() nel padre */
9  #include <sys/wait.h>
10
11 void handle_sigchld(int sig) {
12     (void)sig;
13     int status;
14     pid_t pid;
15     /* WNOHANG: non blocca se nessun figlio e' uscito */
16     while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
17         if (WIFEXITED(status))
18             printf("Figlio %d uscito: %d\n",
19                   pid, WEXITSTATUS(status));
20         else if (WIFSIGNALED(status))
21             printf("Figlio %d ucciso da segnale %d\n",
22                   pid, WTERMSIG(status));
23     }
24 }
25
26 /*
27 * ORFANO: processo il cui padre e' terminato.
28 * Il kernel lo riadotta automaticamente a init/systemd (PID 1).
29 * Non e' un problema: systemd chiama wait() periodicamente.

```

30 `*/`

Attenzione

I processi **zombie** si accumulano se il padre non chiama `wait()`. Non consumano CPU o memoria ma occupano un PID e una entry nella process table. Se si esauriscono i PID (`/proc/sys/kernel/pid_max`, default 4194304), il sistema non può creare nuovi processi.

Capitolo 3

Lo Scheduler del Kernel

Lo scheduler decide quale `task_struct` occupa la CPU in ogni istante. Linux implementa uno scheduler modulare con più *scheduling class*, ciascuna con priorità diverse.

3.1 Classi di scheduling

Tabella 3.1: Classi di scheduling in ordine di priorit  decrescente

Classe	Policy	Caratteristiche
stop_sched_class	—	Solo per CPU hotplug e migration
dl_sched_class	SCHED_DEADLINE	Earliest Deadline First (EDF), hard real-time
rt_sched_class	SCHED_FIFO, SCHED_RR	Real-time, prio 1–99
fair_sched_class	SCHED_NORMAL, SCHED_BATCH	CFS/EEVDF, processi normali
idle_sched_class	SCHED_IDLE	Solo se nessun altro task � pronto

Listing 3.1: Visualizzare e modificare la policy di scheduling

```
1 # Policy e priorit  di un processo
2 chrt -p $(pgrep -n bash)
3 # pid 1234's current scheduling policy: SCHED_OTHER
4 # pid 1234's current scheduling priority: 0
5
6 # Imposta SCHED_FIFO RT con priorit  50
7 sudo chrt -f -p 50 $(pgrep -n bash)
8
9 # Imposta SCHED_DEADLINE (budget 5ms, periodo 10ms)
10 sudo chrt --deadline \
11     --sched-runtime 5000000 \
12     --sched-deadline 10000000 \
13     --sched-period 10000000 \
14     -p 0 $(pgrep -n bash)
15
16 # Visualizza statistiche scheduler per processo
17 cat /proc/$(pgrep -n bash)/sched
18 # se          :      12345.678901 (vruntime in ns)
19 # nr_switches:      9876          (context switch count)
20 # nr_voluntary_switches:      8000
21 # nr_involuntary_switches:    1876
```

3.2 CFS: Completely Fair Scheduler

Il CFS è lo scheduler predefinito per i processi normali (SCHED_NORMAL). Il concetto chiave è il **vruntime** (virtual runtime): ogni task accumula vruntime proporzionalmente al tempo CPU reale, pesato inversamente per la sua priorità (nice value).

Lo scheduler sceglie sempre il task con il **vruntime minore**, usando un albero rosso-nero (*red-black tree*) per mantenere i task ordinati in $O(\log n)$.

Listing 3.2: Struttura sched_entity e vruntime (kernel/sched/fair.c)

```

1  /* Ogni task ha un sched_entity nella sua task_struct */
2  struct sched_entity {
3      struct load_weight    load;          /* peso (funzione del nice) */
4      struct rb_node        run_node;      /* nodo nell'RB-tree della CFS rq */
5      u64                   vruntime;      /* runtime virtuale accumulato */
6      u64                   exec_start;    /* timestamp ultimo context switch */
7      u64                   sum_exec_runtime; /* CPU time reale totale */
8      u64                   prev_sum_exec_runtime;
9      struct cfs_rq         *cfs_rq;      /* run queue CFS a cui appartiene */
10 };
11
12 /*
13  * Aggiornamento vruntime ad ogni tick (kernel/sched/fair.c)
14  * delta_exec = tempo trascorso dall'ultimo update (ns)
15  * weight      = load.weight del task (funzione del nice)
16  * NICE_0_LOAD = peso di un task con nice 0 (= 1024)
17  */
18 static void update_curr(struct cfs_rq *cfs_rq)
19 {
20     struct sched_entity *curr = cfs_rq->curr;
21     u64 now = rq_clock_task(rq_of(cfs_rq));
22     u64 delta_exec;
23
24     if (unlikely(!curr))
25         return;
26
27     delta_exec = now - curr->exec_start;
28     curr->exec_start = now;
29     curr->sum_exec_runtime += delta_exec;
30
31     /* vruntime cresce piu' lentamente per task con basso nice */
32     curr->vruntime += calc_delta_fair(delta_exec, curr);
33     update_min_vruntime(cfs_rq);
34 }
35
36 /*
37  * calc_delta_fair: vruntime_delta = delta_exec * NICE_0_LOAD / weight
38  * Con nice -20 (massima priorita'): weight = 88761 -> vruntime cresce
39  *      lento
40  * Con nice +19 (minima priorita'): weight = 15 -> vruntime cresce
41  *      veloce
42  */

```

Listing 3.3: Nice value e priorita' CFS

```

1  # Mostra nice value di tutti i processi

```

```

2 ps -eo pid,ni,comm | sort -k2 -n | head -20
3
4 # Avvia processo con nice 10 (bassa priorita')
5 nice -n 10 make -j$(nproc)
6
7 # Cambia nice a processo in esecuzione
8 renice -n 5 -p $(pgrep -n make)
9
10 # Peso (weight) in funzione del nice
11 # nice -20 -> weight 88761 (il piu' veloce)
12 # nice 0 -> weight 1024 (default)
13 # nice +19 -> weight 15 (il piu' lento)
14
15 # Verifica vruntime e altri parametri
16 cat /proc/self/sched | grep -E "vruntime|nr_switches|prio"

```

3.3 EEVDF: il nuovo algoritmo dal kernel 6.6

Dal kernel **6.6**, CFS utilizza internamente **EEVDF** (Earliest Eligible Virtual Deadline First) al posto dell'algoritmo leftmost-node per la selezione del prossimo task. EEVDF assegna a ogni task una *virtual deadline* e sceglie il task *eligible* con la deadline più vicina.

Listing 3.4: EEVDF: pick_next_entity dal kernel 6.6+

```

1 /* kernel/sched/fair.c (kernel >= 6.6) */
2 static struct sched_entity *
3 pick_next_entity(struct rq *rq, struct cfs_rq *cfs_rq)
4 {
5     /*
6      * Dal kernel 6.6: pick_eevdf() invece di
7      * __pick_first_entity() (leftmost RB-tree node).
8      *
9      * Un task e' "eligible" se:
10      *   vruntime >= cfs_rq->min_vruntime - lag
11      * dove lag = virtual time non ancora consumato.
12      *
13      * Tra gli eligible, vince chi ha virtual deadline minore.
14      */
15     return pick_eevdf(cfs_rq);
16 }
17
18 /*
19  * Vantaggi EEVDF rispetto a CFS puro:
20  * - Latenza piu' prevedibile (deadline esplicita)
21  * - Migliore fairness in presenza di task con slice variabili
22  * - Fondamenta per sched_ext (BPF schedulers)
23  */

```

3.4 sched_ext: scheduler programmabili con eBPF

Dal kernel **6.12**, **sched_ext** permette di sostituire l'intera logica di selezione del task con programmi eBPF, senza modificare il kernel. Linux 6.19 aggiunge recovery fault-tolerante con riduzione della latenza fino al 15%.

Listing 3.5: Struttura di uno scheduler sched_ext minimale in BPF

```

1  /* my_sched.bpf.c -- scheduler BPF minimale */
2  #include <scx/common.bpf.h>
3
4  /* Struttura dati condivisa tra BPF e userspace */
5  struct {
6      __uint(type, BPF_MAP_TYPE_QUEUE);
7      __uint(max_entries, 4096);
8      __type(value, u32); /* PID dei task */
9  } run_queue SEC(".maps");
10
11  /*
12   * enqueue: chiamato quando un task diventa runnable.
13   * Qui mettiamo il PID in una coda FIFO semplice.
14   */
15  void BPF_STRUCT_OPS(my_enqueue, struct task_struct *p,
16                      u64 enq_flags)
17  {
18      u32 pid = p->pid;
19      scx_bpf_dispatch(p, SCX_DSQ_GLOBAL, SCX_SLICE_DFL,
20                      enq_flags);
21  }
22
23  /* dispatch: selezione del prossimo task */
24  void BPF_STRUCT_OPS(my_dispatch, s32 cpu,
25                      struct task_struct *prev)
26  {
27      scx_bpf_consume(SCX_DSQ_GLOBAL);
28  }
29
30  SEC(".struct_ops")
31  struct sched_ext_ops my_sched_ops = {
32      .enqueue   = (void *)my_enqueue,
33      .dispatch  = (void *)my_dispatch,
34      .name      = "my_minimal_sched",
35  };

```

Listing 3.6: Caricare e usare uno scheduler sched_ext

```

1  # Verifica supporto sched_ext
2  zcat /proc/config.gz | grep CONFIG_SCHED_CLASS_EXT
3  # CONFIG_SCHED_CLASS_EXT=y (kernel 6.12+)
4
5  # Installa scx tools (scheduler collection)
6  # Su Arch:
7  sudo pacman -S scx-scheds
8
9  # Avvia scheduler scx_rustland (Rust-based, bassa latenza)
10 sudo scx_rustland
11
12 # Avvia scheduler scx_layered (cgroups-aware)
13 sudo scx_layered
14
15 # Torna al CFS default: termina lo scheduler BPF
16 # Il kernel fa fallback automatico a CFS/EEVDF
17 sudo pkill scx_rustland
18

```

```

19 # Monitoraggio scheduler
20 # Via /proc
21 cat /proc/schedstat
22
23 # Via perf
24 sudo perf sched record -- sleep 5
25 sudo perf sched latency | head -30
26
27 # Via ftrace (latenza wakeup)
28 echo 1 > /sys/kernel/debug/tracing/events/sched/enable
29 cat /sys/kernel/debug/tracing/trace | head -40

```

3.5 Real-time scheduling

Listing 3.7: SCHED_FIFO e SCHED_RR per task real-time

```

1 #include <sched.h>
2 #include <string.h>
3
4 int set_realtime(int policy, int priority) {
5     struct sched_param param;
6     memset(&param, 0, sizeof(param));
7     param.sched_priority = priority; /* 1-99, 99=massima */
8
9     /*
10      * SCHED_FIFO: FIFO pura, non cede la CPU
11      *           finche' non si blocca o cede volontariamente
12      * SCHED_RR: Round-Robin con quantum fisso (RR_INTERVAL)
13      *           tra task con stessa priorita'
14      */
15     return sched_setscheduler(0, policy, &param);
16 }
17
18 /* Impostare la CPU affinity */
19 #include <pthread.h>
20
21 void pin_to_cpu(int cpu) {
22     cpu_set_t cpuset;
23     CPU_ZERO(&cpuset);
24     CPU_SET(cpu, &cpuset);
25     pthread_setaffinity_np(pthread_self(),
26                             sizeof(cpuset), &cpuset);
27 }

```

Listing 3.8: Tuning scheduler e diagnostica

```

1 # Verifica quantum RR (in ms)
2 cat /proc/sys/kernel/sched_rr_timeslice_ms
3
4 # Disabilita migrazione automatica tra CPU (per RT)
5 echo -1 > /proc/sys/kernel/sched_rt_runtime_us
6
7 # Mostra statistiche per ogni CPU
8 mpstat -P ALL 1 3
9

```

```
10 # Mostra latenza scheduling con trace-cmd
11 sudo trace-cmd record -e sched_switch sleep 1
12 sudo trace-cmd report | head -30
13
14 # CPU isolation (dedicare CPU a task RT)
15 # In /etc/default/grub: GRUB_CMDLINE_LINUX="isolcpus=2,3"
16 # Poi mettere il processo sulle CPU isolate:
17 taskset -c 2 sudo chrt -f 90 ./mio_rt_task
```

Suggerimento

Per misurare la latenza di scheduling in modo preciso, usare `cyclicttest` dal pacchetto `rt-tests`:

```
sudo apt install rt-tests # Debian/Kali
sudo pacman -S rt-tests   # Arch
sudo cyclicttest -l 100000 -m -n -q -p95 -i200
# Max latency: < 50 us su sistema ben configurato
```

Capitolo 4

Gestione della Memoria

La gestione della memoria è uno dei sottosistemi più complessi del kernel Linux. Il codice principale si trova in `mm/` e gestisce la memoria fisica, le pagine virtuali, gli allocatori e lo swap.

4.1 Memoria fisica e pagine

Il kernel suddivide la RAM fisica in **pagine** di 4 KB (su `x86_64`). Ogni pagina fisica è descritta da una `struct page` in `include/linux/mm_types.h`.

Listing 4.1: `struct page` e gestione pagine fisiche

```
1  /* include/linux/mm_types.h (semplificato) */
2  struct page {
3      unsigned long flags;          /* PG_locked, PG_dirty, PG_uptodate... */
4      union {
5          struct {                  /* pagina in uso da un processo */
6              struct list_head lru; /* lista LRU */
7              struct address_space *mapping;
8              pgoff_t index;
9          };
10         struct {                   /* pagina dello slab allocator */
11             struct slab *slab_head;
12             void *freelist;
13         };
14         struct {                   /* pagina composta (huge page) */
15             unsigned long compound_head;
16         };
17     };
18     atomic_t _refcount;            /* contatore riferimenti */
19     /* ... altri campi omessi */
20 };
21
22 /* Conversioni tra indirizzo fisico e struct page */
23 /* phys -> page */
24 struct page *page = pfn_to_page(phys >> PAGE_SHIFT);
25 /* page -> phys */
26 unsigned long phys = page_to_phys(page);
27 /* page -> indirizzo virtuale kernel */
28 void *vaddr = page_address(page);
```

Listing 4.2: Informazioni memoria fisica

```
1  # Memoria fisica totale e disponibile
2  free -h
3  cat /proc/meminfo | head -20
4
5  # Zone di memoria (DMA, Normal, HighMem)
6  cat /proc/zoneinfo | grep -E "^Node|^zone|present|managed"
```

```

7
8 # Statistiche paginator (buddy system)
9 cat /proc/buddyinfo
10 # Node 0, zone DMA      : 1 0 1 0 2 1 1 0 1 1 3
11 # Node 0, zone Normal   : 4231 2890 1245 678 ... (ordine 0..10)
12
13 # Uso delle huge pages
14 cat /proc/meminfo | grep Huge
15 # AnonHugePages:      204800 kB
16 # HugePages_Total:    0
17 # HugePages_Free:     0
18 # Hugepagesize:       2048 kB
19
20 # NUMA topology
21 numactl --hardware
22 cat /proc/buddyinfo

```

4.2 Buddy System e allocazione pagine

Il kernel alloca pagine fisiche tramite il **Buddy System** (`mm/page_alloc.c`), un allocatore che gestisce blocchi di pagine di dimensione 2^n (ordini da 0 a 10, cioè 1 a 1024 pagine).

Listing 4.3: Allocazione pagine nel kernel

```

1 /* Alloca pagine dal kernel */
2 #include <linux/gfp.h>
3
4 /* Alloca 1 pagina (2^0 = 4 KB) */
5 struct page *page = alloc_page(GFP_KERNEL);
6
7 /* Alloca 2^order pagine (es. order=2 -> 16 KB) */
8 struct page *pages = alloc_pages(GFP_KERNEL, 2);
9
10 /* Libera pagine */
11 __free_page(page);
12 __free_pages(pages, 2);
13
14 /*
15  * Flag GFP (Get Free Pages):
16  * GFP_KERNEL : puo' dormire, usa normale memoria
17  * GFP_ATOMIC : non puo' dormire (interrupt handler)
18  * GFP_DMA    : solo memoria DMA (<16MB su x86)
19  * GFP_HIGHMEM : puo' usare high memory
20  * __GFP_ZERO  : azzera la pagina prima di restituirla
21  * __GFP_NOWARN: non stampa warning se alloc fallisce
22  */
23 struct page *zeroed = alloc_page(GFP_KERNEL | __GFP_ZERO);

```

4.3 Slab Allocator: SLUB

Per allocazioni di oggetti di dimensione fissa (es. `task_struct`, `inode`, `dentry`), il kernel usa lo **SLUB allocator** (`mm/slub.c`), che mantiene cache di oggetti pre-allocati e inizializzati.

Listing 4.4: `kmalloc` e `kmem_cache`

```

1  /* Allocazione generica in kernel space */
2  #include <linux/slab.h>
3
4  /* kcalloc: alloca in byte, simile a malloc userspace */
5  void *ptr = kcalloc(256, GFP_KERNEL);
6  void *zptr = kzalloc(256, GFP_KERNEL); /* + azzera memoria */
7  kfree(ptr);
8
9  /* kcalloc: array di n elementi di size byte */
10 struct mia_struttura *arr = kcalloc(100,
11     sizeof(struct mia_struttura), GFP_KERNEL);
12 kfree(arr);
13
14 /* Cache dedicata per oggetti frequenti */
15 static struct kmem_cache *mia_cache;
16
17 /* Inizializzazione (una volta, al caricamento modulo) */
18 mia_cache = kmem_cache_create(
19     "mia_struttura",          /* nome (visibile in /proc/slabinfo) */
20     sizeof(struct mia_struttura), /* dimensione oggetto */
21     0,                        /* allineamento (0=default) */
22     SLAB_HWCACHE_ALIGN,      /* flag */
23     NULL                      /* ctor */
24 );
25
26 /* Uso: molto piu' veloce di kcalloc per oggetti ripetuti */
27 struct mia_struttura *obj = kmem_cache_alloc(mia_cache,
28     GFP_KERNEL);
29 kmem_cache_free(mia_cache, obj);
30
31 /* Cleanup (al rimozione modulo) */
32 kmem_cache_destroy(mia_cache);

```

Listing 4.5: Diagnostica SLUB

```

1  # Statistiche slab (oggetti attivi, per cache)
2  cat /proc/slabinfo | head -20
3
4  # Name          <active_objs> <num_objs> <objsize> ...
5  # task_struct    523          580      7424 ...
6  # dentry         12840        13020     192 ...
7  # inode_cache    3456         3600      648 ...
8
9  # Alternativa moderna (piu' leggibile)
10 sudo slabtop -o | head -20
11
12 # Uso totale memoria slab
13 cat /proc/meminfo | grep Slab
14
15 # Dettaglio per cache specifica
16 cat /sys/kernel/slab/task_struct/object_size
17 cat /sys/kernel/slab/task_struct/slabs

```

4.4 Memoria virtuale e VMA

Ogni processo ha il proprio spazio di indirizzamento virtuale, descritto da `struct mm_struct` (`mm/mm_types.h`). Lo spazio virtuale è suddiviso in **VMA** (Virtual Memory Areas): `struct vm_area_struct`.

Listing 4.6: `struct mm_struct` e `vm_area_struct`

```

1 struct mm_struct {
2     struct maple_tree mm_mt;    /* albero delle VMA (kernel 6.1+) */
3     unsigned long      mmap_base;
4     unsigned long      task_size; /* limite user space */
5     pgd_t              *pgd;    /* Page Global Directory (tabelle pagine)
6     /*
7     atomic_t           mm_count;
8     atomic_t           mm_users;
9     unsigned long      total_vm; /* pagine virtuali totali */
10    unsigned long      locked_vm; /* pagine bloccate in RAM */
11    unsigned long      data_rlimit;
12    /* Sezioni del processo */
13    unsigned long      start_code, end_code; /* .text */
14    unsigned long      start_data, end_data; /* .data */
15    unsigned long      start_brk, brk;      /* heap */
16    unsigned long      start_stack;         /* stack */
17    unsigned long      mmap_legacy_base;
18 };
19
20 struct vm_area_struct {
21     unsigned long      vm_start; /* indirizzo virtuale inizio */
22     unsigned long      vm_end;   /* indirizzo virtuale fine */
23     struct mm_struct    *vm_mm;
24     pgprot_t           vm_page_prot; /* protezione (R/W/X) */
25     unsigned long      vm_flags;    /* VM_READ|VM_WRITE|VM_EXEC... */
26     struct file         *vm_file;   /* file mappato (NULL per anonima) */
27     unsigned long      vm_pgoff;    /* offset nel file */
28     const struct vm_operations_struct *vm_ops; /* fault handler */
29 };

```

Listing 4.7: Esaminare la mappa di memoria di un processo

```

1 # Mappa virtuale del processo corrente
2 cat /proc/self/maps
3
4 # Formato: start-end perms offset dev inode [pathname]
5 # 55e8a0000000-55e8a0001000 r--p 00000000 08:01 123456 /bin/cat
6 # 55e8a0001000-55e8a0002000 r-xp 00001000 08:01 123456 /bin/cat (.text)
7 # 7ffd1234f000-7ffd12370000 rwxp 00000000 00:00 0 [stack]
8 # 7ffd1238d000-7ffd12391000 r--p 00000000 00:00 0 [vvar]
9 # 7ffd12391000-7ffd12393000 r-xp 00000000 00:00 0 [vdso]
10
11 # Formato esteso (con RSS per VMA)
12 cat /proc/self/smaps | head -40
13
14 # Statistiche sommario memoria processo
15 cat /proc/self/smaps_rollup
16
17 # Usa pmap (dal pacchetto procps)
18 pmap -x $(pgrep -n bash)

```

4.5 Page fault e Copy-on-Write

Quando un processo accede a una pagina non presente in RAM, la CPU genera un **page fault** (interrupt #14 su x86). Il kernel gestisce il fault in `handle_mm_fault()`:

Listing 4.8: Gestione page fault (mm/memory.c)

```

1  /* Tipi di page fault */
2  /*
3   * DEMAND PAGING: la pagina non e' mai stata caricata
4   *   -> kernel legge dal file/swap e la mappa
5   *
6   * COPY-ON-WRITE (COW): fork() ha condiviso la pagina
7   *   -> al primo write, kernel crea copia privata
8   *
9   * STACK GROWTH: accesso appena sotto lo stack
10  *   -> kernel espande automaticamente la VMA stack
11  *
12  * SEGFault: accesso non autorizzato
13  *   -> SIGSEGV al processo
14  */
15
16  vm_fault_t handle_mm_fault(struct vm_area_struct *vma,
17                             unsigned long address,
18                             unsigned int flags,
19                             struct pt_regs *regs)
20  {
21      /* 1. Verifica permessi della VMA */
22      /* 2. Cammina le page table */
23      /* 3. Se PTE presente: COW o protezione */
24      /* 4. Se PTE assente: do_fault() */
25      /*   - do_read_fault: copia da file */
26      /*   - do_cow_fault: copia privata */
27      /*   - do_shared_fault: mapping condiviso */
28      return __handle_mm_fault(vma, address, flags);
29  }

```

Listing 4.9: Statistiche page fault e memoria

```

1  # Page fault del processo
2  /usr/bin/time -v ls /etc 2>&1 | grep "Page size\|Major\|Minor"
3  # Major (I/O) page faults: 0
4  # Minor (reclaim) page faults: 234
5
6  # Statistiche kernel-wide
7  cat /proc/vmstat | grep -E "pgfault|pgmajfault"
8  # pgfault      1234567      (minor faults totali)
9  # pgmajfault   12345      (major faults totali)
10
11  # Pressione memoria e OOM killer
12  dmesg | grep -i "oom\|killed process" | tail -10
13
14  # Impostare swappiness (0=evita swap, 100=usa swap aggressivamente)
15  cat /proc/sys/vm/swappiness # default: 60
16  sudo sysctl vm.swappiness=10 # valori bassi su desktop
17
18  # Transparent Huge Pages

```

```
19 cat /sys/kernel/mm/transparent_hugepage/enabled
20 # [always] madvise never
21
22 # Abilita THP solo su richiesta (madvise)
23 echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

4.6 OOM Killer

Quando il sistema esaurisce la memoria, il kernel invoca l'**OOM Killer** (Out Of Memory Killer) che sceglie un processo da terminare basandosi su un punteggio (*oom_score*).

Listing 4.10: OOM Killer: configurazione e diagnostica

```
1 # Punteggio OOM di ogni processo (0-1000)
2 cat /proc/self/oom_score
3
4 # Aggiustamento manuale (da -1000 a +1000)
5 # -1000 = mai terminare, +1000 = termina per primo
6 echo -500 | sudo tee /proc/1/oom_score_adj # proteggi init
7 echo 500 | sudo tee /proc/$(pgrep -n chrome)/oom_score_adj
8
9 # Disabilita OOM killer completamente (pericoloso!)
10 # sudo sysctl vm.overcommit_memory=2
11 # sudo sysctl vm.overcommit_ratio=80
12
13 # Politiche di overcommit
14 # 0 = heuristic (default)
15 # 1 = sempre permetti (nessun controllo)
16 # 2 = non permettere overcommit
17 cat /proc/sys/vm/overcommit_memory
18
19 # Log OOM killer
20 journalctl -k | grep -i "oom\|memory" | tail -20
21 dmesg -T | grep -i "oom killer\|killed process"
```

Capitolo 5

VFS: Virtual Filesystem Switch

Il VFS è lo strato di astrazione che permette al kernel di supportare decine di filesystem diversi (ext4, xfs, btrfs, tmpfs, proc...) con un'interfaccia unificata.

5.1 Strutture dati fondamentali

Listing 5.1: Le quattro strutture chiave del VFS

```
1  /*
2  * 1. SUPERBLOCK: rappresenta un filesystem montato
3  *   (una istanza per ogni mount)
4  */
5  struct super_block {
6      dev_t          s_dev;          /* device id */
7      unsigned long  s_blocksize;    /* dimensione blocco */
8      unsigned char  s_blocksize_bits;
9      loff_t         s_maxbytes;     /* dimensione max file */
10     struct file_system_type *s_type;
11     const struct super_operations *s_op; /* operazioni */
12     unsigned long   s_flags;        /* MS_RDONLY, MS_NOSUID... */
13     struct dentry   *s_root;        /* dentry radice */
14     /* ... */
15 };
16
17 /*
18 * 2. INODE: metadati di un file (permessi, owner, size...)
19 *   Indipendente dal nome; piu' nomi (hard link) -> stesso inode
20 */
21 struct inode {
22     umode_t         i_mode;         /* tipo e permessi (rwxrwxrwx) */
23     kuid_t          i_uid;         /* UID proprietario */
24     kgid_t          i_gid;         /* GID proprietario */
25     unsigned int     i_nlink;       /* numero hard link */
26     loff_t          i_size;         /* dimensione in byte */
27     struct timespec64 i_atime, i_mtime, i_ctime;
28     unsigned long    i_ino;         /* numero inode */
29     const struct inode_operations *i_op; /* mkdir, link... */
30     const struct file_operations *i_fop; /* read, write... */
31     struct address_space *i_mapping; /* page cache */
32 };
33
34 /*
35 * 3. DENTRY: componente di percorso (directory entry)
36 *   Associa nome <-> inode; fortemente cached (dcache)
37 */
38 struct dentry {
39     unsigned int     d_flags;
40     struct inode     *d_inode;      /* inode associato */

```

```

41     struct dentry    *d_parent;      /* directory padre */
42     struct qstr      d_name;         /* nome componente */
43     struct list_head d_child;        /* nodi fratelli */
44     struct list_head d_subdirs;      /* figli */
45     struct dentry_operations *d_op;
46 };
47
48 /*
49  * 4. FILE: istanza aperta di un file da un processo
50  *      Una per ogni open(); piu' FILE possono puntare allo stesso inode
51  */
52 struct file {
53     struct path      f_path;         /* dentry + vfsmount */
54     struct inode      *f_inode;
55     const struct file_operations *f_op;
56     loff_t           f_pos;          /* posizione corrente */
57     unsigned int      f_flags;        /* O_RDONLY, O_WRONLY... */
58     fmode_t           f_mode;         /* FMODE_READ, FMODE_WRITE */
59     struct fown_struct f_owner;       /* per SIGIO */
60     void              *private_data; /* uso del driver */
61 };

```

Listing 5.2: Esaminare strutture VFS da spazio utente

```

1  # Informazioni inode
2  stat /etc/passwd
3  # File: /etc/passwd
4  # Size: 2345      Blocks: 8      IO Block: 4096   regular file
5  # Device: 802h/2050d Inode: 131075 Links: 1
6  # Access: (0644/-rw-r--r--)  Uid: (0/root)   Gid: (0/root)
7
8  # Mostra numero inode nella lista directory
9  ls -li /etc/ | head -10
10 # 131075 -rw-r--r-- 1 root root 2345 gen 10 10:00 passwd
11
12 # Inode liberi sul filesystem
13 df -i /
14
15 # File aperti (= struct file nel kernel)
16 lsof -p $$ | head -10 # file aperti dalla shell corrente
17
18 # File descriptor della shell corrente
19 ls -la /proc/self/fd/
20
21 # Dentry cache statistics
22 cat /proc/sys/fs/dentry-state
23 # 156789 145678 45 0 0 0
24 # (nr_dentry, nr_unused, age_limit, want_pages, nr_negative, dummy)
25
26 # Inode cache
27 cat /proc/sys/fs/inode-state

```

5.2 File operations: l'interfaccia driver-VFS

Listing 5.3: struct file_operations — il cuore di ogni driver/fs

```

1  /* include/linux/fs.h */
2  struct file_operations {
3      struct module *owner;
4      loff_t  (*llseek)  (struct file *, loff_t, int);
5      ssize_t (*read)    (struct file *, char __user *, size_t, loff_t *);
6      ssize_t (*write)   (struct file *, const char __user *, size_t, loff_t
7                          *);
8      int     (*open)    (struct inode *, struct file *);
9      int     (*release) (struct inode *, struct file *);
10     int     (*fsync)    (struct file *, loff_t, loff_t, int datasync);
11     long    (*unlocked_ioctl)(struct file *, unsigned int, unsigned long);
12     int     (*mmap)     (struct file *, struct vm_area_struct *);
13     /* ... molti altri campi */
14 };
15
16 /* Esempio: implementazione minima per un char device */
17 static ssize_t mio_read(struct file *file,
18                        char __user *buf,
19                        size_t count, loff_t *ppos)
20 {
21     static const char msg[] = "Ciao dal kernel!\n";
22     size_t len = min(count, sizeof(msg) - *ppos);
23
24     if (len == 0)
25         return 0;
26
27     /* copy_to_user: copia sicura kernel -> user space */
28     if (copy_to_user(buf, msg + *ppos, len))
29         return -EFAULT;
30
31     *ppos += len;
32     return len;
33 }
34
35 static const struct file_operations mio_fops = {
36     .owner  = THIS_MODULE,
37     .read   = mio_read,
38     .llseek = default_llseek,
39 };

```

5.3 Page cache e writeback

Tutte le letture da disco passano attraverso la **page cache**: se la pagina è già in RAM, non serve accedere al disco (cache hit). Le scritture sono asincrone (writeback).

Listing 5.4: Page cache: statistiche e gestione

```

1  # Dimensione page cache
2  cat /proc/meminfo | grep -E "^Cached:|^Buffers:|^Dirty:|^Writeback:"
3  # Buffers:          123456 kB   (buffer I/O raw block)
4  # Cached:           2345678 kB  (page cache file)
5  # Dirty:            12345 kB   (pagine da scrivere su disco)
6  # Writeback:        0 kB      (in fase di scrittura)
7
8  # Forzare flush page cache su disco

```

```
9 sync
10
11 # Svuotare la page cache (per test performance)
12 echo 3 | sudo tee /proc/sys/vm/drop_caches
13 # 1 = page cache, 2 = dentries+inodes, 3 = tutto
14
15 # Parametri writeback
16 cat /proc/sys/vm/dirty_ratio # % RAM prima di bloccare
17 cat /proc/sys/vm/dirty_background_ratio # % RAM per avviare writeback
18 cat /proc/sys/vm/dirty_expire_centisecs # eta' max pagine dirty (cs)
19
20 # Monitoraggio writeback
21 iostat -x 1 | grep -v "^$" | head -10
22 iotop -ao # I/O per processo (Arch: sudo pacman -S iotop)
```

5.4 Filesystem principali

Tabella 5.1: Filesystem Linux a confronto tecnico

FS	Journal	Caratteristiche	Usato su
ext4	Full journal	Estents, dir_index, max 16TB	Debian default, root
xfs	Metadata journal	Scalabile, ottimo per grandi file	Red Hat default
btrfs	Copy-on-write	Snapshot, subvol, RAID integrato	Arch, Fedora
tmpfs	Nessuno	Solo RAM, velocissimo	/tmp, /dev/shm
proc	Nessuno	Pseudo-fs kernel info	/proc
sysfs	Nessuno	Pseudo-fs hardware	/sys
overlayfs	Dipende	Union mount (layers)	Docker, containers

Listing 5.5: Operazioni avanzate sui filesystem

```
1 # --- ext4 ---
2 # Informazioni superbblock
3 sudo tune2fs -l /dev/sda1 | grep -E "Block count|Inode count|Journal"
4
5 # Verifica filesystem (offline)
6 sudo e2fsck -f /dev/sdb1
7
8 # --- btrfs ---
9 # Lista subvolume
10 sudo btrfs subvolume list /
11
12 # Crea snapshot (istantaneo, copy-on-write)
13 sudo btrfs subvolume snapshot / /snapshots/root-$(date +%Y%m%d)
14
15 # Uso disco per subvolume
16 sudo btrfs filesystem du -s /home
17
```

```

18 # Bilanciamento RAID btrfs
19 sudo btrfs balance start -dconvert=raid1 -mconvert=raid1 /
20
21 # --- overlayfs (come Docker) ---
22 mkdir -p /tmp/lower /tmp/upper /tmp/work /tmp/merged
23 echo "file originale" > /tmp/lower/test.txt
24 sudo mount -t overlay overlay \
25   -o lowerdir=/tmp/lower,upperdir=/tmp/upper,workdir=/tmp/work \
26   /tmp/merged
27 # /tmp/merged vede lower; modifiche vanno in upper

```

5.5 /proc e /sys: filesystem virtuali

Listing 5.6: Navigare /proc e /sys per diagnostica kernel

```

1 # /proc: informazioni kernel e processi
2 cat /proc/cpuinfo           # CPU info (per core)
3 cat /proc/interrupts        # interrupt per CPU e tipo
4 cat /proc/net/dev           # statistiche interfacce di rete
5 cat /proc/net/tcp           # connessioni TCP (hex)
6 cat /proc/sys/kernel/       # parametri kernel modificabili
7
8 # Modifica parametri kernel via sysctl
9 sudo sysctl -w kernel.sysrq=1           # abilita magic SysRq
10 sudo sysctl -w net.core.rmem_max=26214400 # buffer socket
11
12 # Rende permanente (aggiunge a /etc/sysctl.d/)
13 echo "net.core.rmem_max=26214400" | \
14   sudo tee /etc/sysctl.d/99-net.conf
15 sudo sysctl -p /etc/sysctl.d/99-net.conf
16
17 # /sys: hardware e driver
18 ls /sys/bus/pci/devices/         # dispositivi PCI
19 cat /sys/block/sda/queue/scheduler # I/O scheduler
20 echo mq-deadline | sudo tee /sys/block/sda/queue/scheduler
21
22 # Frequenza CPU e governor
23 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
24 echo performance | sudo tee \
25   /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
26
27 # Power management
28 cat /sys/power/state             # stati supportati: freeze mem disk

```

Capitolo 6

Driver e Moduli Kernel

I moduli kernel (file `.ko`) permettono di estendere il kernel a runtime senza ricompilazione. Sono caricabili e scaricabili dinamicamente tramite `insmod/rmmod` o il più intelligente `modprobe`.

6.1 Anatomia di un modulo kernel

Listing 6.1: Modulo kernel minimale (hello.c)

```
1  /* hello.c -- modulo kernel minimale */
2  #include <linux/module.h>      /* MODULE_* macro */
3  #include <linux/kernel.h>      /* pr_info, KERN_INFO */
4  #include <linux/init.h>        /* __init, __exit */
5
6  MODULE_LICENSE("GPL");
7  MODULE_AUTHOR("OTHack1A");
8  MODULE_DESCRIPTION("Modulo di esempio");
9  MODULE_VERSION("1.0");
10
11 /* __init: il codice viene liberato dalla memoria dopo l'init */
12 static int __init hello_init(void)
13 {
14     pr_info("hello: modulo caricato, kernel %s\n",
15            utsname()->release);
16     return 0; /* 0 = successo, negativo = errore */
17 }
18
19 /* __exit: ignorato se il modulo e' compilato nel kernel */
20 static void __exit hello_exit(void)
21 {
22     pr_info("hello: modulo rimosso\n");
23 }
24
25 module_init(hello_init);
26 module_exit(hello_exit);
```

Listing 6.2: Makefile e compilazione modulo

```
1  # Makefile per modulo out-of-tree
2  obj-m += hello.o
3
4  KDIR := /lib/modules/$(shell uname -r)/build
5  PWD  := $(shell pwd)
6
7  all:
8      make -C $(KDIR) M=$(PWD) modules
9
10 clean:
11     make -C $(KDIR) M=$(PWD) clean
```

```

12
13 # Compilazione
14 make
15
16 # Carica il modulo
17 sudo insmod hello.ko
18
19 # Verifica
20 lsmod | grep hello
21 dmesg | tail -3
22 # [12345.678] hello: modulo caricato, kernel 6.1.0-20-amd64
23
24 # Rimuovi
25 sudo rmmod hello
26 dmesg | tail -2
27 # [12350.123] hello: modulo rimosso
28
29 # modprobe: gestisce dipendenze automaticamente
30 sudo modprobe hello          # cerca in /lib/modules/$(uname -r)/
31 sudo modprobe -r hello       # rimuovi
32 sudo modprobe -v hello       # verbose

```

6.2 Modulo con parametri

Listing 6.3: Parametri del modulo tramite module_param

```

1 #include <linux/module.h>
2 #include <linux/moduleparam.h>
3
4 static int debug_level = 0;
5 static char *device_name = "mio_device";
6 static int buffer_size = 4096;
7
8 /* Tipi supportati: bool, byte, short, ushort, int, uint,
9  *                long, ulong, charp, array */
10 module_param(debug_level, int, 0644); /* lettura/scrittura da root */
11 module_param(device_name, charp, 0444); /* sola lettura */
12 module_param(buffer_size, int, 0644);
13
14 MODULE_PARM_DESC(debug_level, "Livello di debug (0-3)");
15 MODULE_PARM_DESC(device_name, "Nome del device");
16 MODULE_PARM_DESC(buffer_size, "Dimensione buffer in byte");
17
18 static int __init mio_init(void)
19 {
20     pr_info("debug=%d device=%s buf=%d\n",
21            debug_level, device_name, buffer_size);
22     return 0;
23 }
24 module_init(mio_init);
25 MODULE_LICENSE("GPL");

```

Listing 6.4: Passare parametri al modulo

```

1 # Al caricamento

```

```

2  sudo modprobe mio_modulo debug_level=2 buffer_size=8192
3
4  # Parametri visibili in /sys
5  ls /sys/module/mio_modulo/parameters/
6  cat /sys/module/mio_modulo/parameters/debug_level
7
8  # Modifica a runtime (se 0644)
9  echo 3 | sudo tee /sys/module/mio_modulo/parameters/debug_level
10
11 # Parametri permanenti via /etc/modprobe.d/
12 echo "options mio_modulo debug_level=1" | \
13     sudo tee /etc/modprobe.d/mio_modulo.conf

```

6.3 Character device driver

Listing 6.5: Driver char device completo

```

1  #include <linux/module.h>
2  #include <linux/fs.h>
3  #include <linux/cdev.h>
4  #include <linux/device.h>
5  #include <linux/uaccess.h>
6
7  #define DEVICE_NAME "mydev"
8  #define BUF_SIZE    256
9
10 static dev_t      dev_num;          /* major:minor allocato */
11 static struct cdev my_cdev;
12 static struct class *my_class;
13 static char       kbuf[BUF_SIZE];
14 static size_t     kbuf_len = 0;
15
16 static int mydev_open(struct inode *inode, struct file *file)
17 {
18     pr_info("mydev: open (pid=%d)\n", current->pid);
19     return 0;
20 }
21
22 static int mydev_release(struct inode *inode, struct file *file)
23 {
24     pr_info("mydev: close\n");
25     return 0;
26 }
27
28 static ssize_t mydev_read(struct file *file, char __user *buf,
29                          size_t count, loff_t *ppos)
30 {
31     size_t available = kbuf_len - *ppos;
32     size_t to_copy = min(count, available);
33
34     if (to_copy == 0)
35         return 0; /* EOF */
36
37     if (copy_to_user(buf, kbuf + *ppos, to_copy))
38         return -EFAULT;
39

```

```

40     *ppos += to_copy;
41     return to_copy;
42 }
43
44 static ssize_t mydev_write(struct file *file,
45                           const char __user *buf,
46                           size_t count, loff_t *ppos)
47 {
48     size_t to_copy = min(count, (size_t)BUF_SIZE - 1);
49
50     if (copy_from_user(kbuf, buf, to_copy))
51         return -EFAULT;
52
53     kbuf[to_copy] = '\0';
54     kbuf_len = to_copy;
55     return to_copy;
56 }
57
58 static const struct file_operations mydev_fops = {
59     .owner    = THIS_MODULE,
60     .open     = mydev_open,
61     .release  = mydev_release,
62     .read     = mydev_read,
63     .write    = mydev_write,
64 };
65
66 static int __init mydev_init(void)
67 {
68     int ret;
69
70     /* Alloca major number dinamicamente */
71     ret = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
72     if (ret < 0) return ret;
73
74     /* Inizializza e registra il cdev */
75     cdev_init(&my_cdev, &mydev_fops);
76     my_cdev.owner = THIS_MODULE;
77     ret = cdev_add(&my_cdev, dev_num, 1);
78     if (ret < 0) goto err_cdev;
79
80     /* Crea /dev/mydev automaticamente via udev */
81     my_class = class_create(THIS_MODULE, DEVICE_NAME);
82     if (IS_ERR(my_class)) { ret = PTR_ERR(my_class); goto err_class; }
83
84     device_create(my_class, NULL, dev_num, NULL, DEVICE_NAME);
85     pr_info("mydev: registrato su %d:%d\n",
86           MAJOR(dev_num), MINOR(dev_num));
87     return 0;
88
89 err_class: cdev_del(&my_cdev);
90 err_cdev:  unregister_chrdev_region(dev_num, 1);
91     return ret;
92 }
93
94 static void __exit mydev_exit(void)
95 {
96     device_destroy(my_class, dev_num);

```

```

97     class_destroy(my_class);
98     cdev_del(&my_cdev);
99     unregister_chrdev_region(dev_num, 1);
100 }
101
102 module_init(mydev_init);
103 module_exit(mydev_exit);
104 MODULE_LICENSE("GPL");

```

Listing 6.6: Test del char device

```

1  # Carica il driver
2  sudo insmod mydev.ko
3
4  # Verifica creazione /dev/mydev
5  ls -la /dev/mydev
6  # crw----- 1 root root 245, 0 ... /dev/mydev
7
8  # Scrivi e leggi
9  echo "ciao kernel" | sudo tee /dev/mydev
10 sudo cat /dev/mydev
11 # ciao kernel
12
13 # Cambia permessi per test utente normale
14 sudo chmod 666 /dev/mydev
15 echo "test" > /dev/mydev
16 cat /dev/mydev
17
18 # Monitoraggio con dmesg
19 dmesg -w | grep mydev

```

6.4 Dipendenze tra moduli e blacklist

Listing 6.7: Gestione avanzata moduli

```

1  # Dipendenze di un modulo
2  modinfo bluetooth | grep depends
3  modinfo snd_hda_intel | grep depends
4
5  # Albero delle dipendenze
6  sudo modprobe --show-depends bluetooth
7
8  # Blocca un modulo (es: driver grafico proprietario vs nouveau)
9  echo "blacklist nouveau" | \
10     sudo tee /etc/modprobe.d/blacklist-nouveau.conf
11 echo "options nouveau modeset=0" | \
12     sudo tee -a /etc/modprobe.d/blacklist-nouveau.conf
13
14 # Aggiorna initramfs dopo blacklist
15 sudo update-initramfs -u          # Debian/Kali
16 sudo mkinitcpio -P               # Arch Linux
17
18 # Forza alias modulo (es: usa driver specifico per device)
19 echo "alias pci:v00008086d00002526* i915" | \
20     sudo tee /etc/modprobe.d/i915.conf

```

```
21
22 # Informazioni complete su modulo
23 modinfo ext4
24 # filename:          /lib/modules/6.1.../kernel/fs/ext4/ext4.ko
25 # description:       Fourth Extended Filesystem
26 # author:            Remy Card...
27 # license:            GPL
28 # vermagic:          6.1.0-20-amd64 SMP preempt mod_unload modversions
```

Capitolo 7

Sottosistema di Rete

Il network stack di Linux è un'implementazione completa del modello TCP/IP, contenuta in `net/`. È composto da oltre 500.000 righe di codice e supporta centinaia di protocolli.

7.1 Architettura dello stack di rete

Listing 7.1: Struttura `sk_buff`: il pacchetto nel kernel

```
1  /*
2   * sk_buff (socket buffer) e' il contenitore
3   * di ogni pacchetto di rete nel kernel Linux.
4   * Definito in include/linux/skbuff.h
5   */
6  struct sk_buff {
7      /* Lista e scheduling */
8      struct sk_buff    *next, *prev;
9      struct sock        *sk;      /* socket proprietaria */
10     struct net_device   *dev;     /* interfaccia di rete */
11
12     /* Puntatori ai layer del protocollo */
13     unsigned char       *head;    /* inizio buffer allocato */
14     unsigned char       *data;    /* inizio dati corrente */
15     unsigned char       *tail;    /* fine dati corrente */
16     unsigned char       *end;     /* fine buffer allocato */
17
18     /* Header layer (impostati man mano che si sale lo stack) */
19     unsigned char       *transport_header; /* TCP/UDP */
20     unsigned char       *network_header;   /* IP */
21     unsigned char       *mac_header;       /* Ethernet */
22
23     /* Metadati */
24     unsigned int        len;           /* lunghezza dati */
25     unsigned int        data_len;      /* len dati non-lineari */
26     __u16               protocol;      /* ETH_P_IP, ETH_P_IPV6... */
27     __u8               pkt_type;      /* PACKET_HOST, BROADCAST... */
28     /* checksum, timestamps, conntrack info, ... */
29 };
30
31 /* Operazioni comuni su sk_buff */
32 struct sk_buff *skb = alloc_skb(1500, GFP_KERNEL);
33 skb_put(skb, 100);           /* aggiunge bytes alla fine */
34 skb_push(skb, ETH_HLEN);     /* aggiunge header in cima */
35 skb_pull(skb, ETH_HLEN);     /* rimuove header */
36 kfree_skb(skb);             /* libera il buffer */
```

Tabella 7.1: Percorso di un pacchetto in ricezione

#	Layer	Azione
1	Hardware/NIC	DMA frame in memoria, IRQ al kernel
2	Driver NIC	Alloca skb, copia frame, chiama netif_rx()
3	NAPI/softirq	net_rx_action() processa coda skb
4	Layer 2 (ETH)	eth_type_trans(), controlla MAC, protocollo
5	Netfilter	Hook PREROUTING (iptables/nftables)
6	Layer 3 (IP)	ip_rcv(), routing, TTL, defrag
7	Netfilter	Hook INPUT (iptables/nftables)
8	Layer 4 (TCP/UDP)	tcp_v4_rcv(), gestione connessione
9	Socket buffer	skb in receive queue del socket
10	User space	recvmsg() sveglia il processo

7.2 Socket e comunicazione di rete

Listing 7.2: Server TCP minimale in C (syscall dirette)

```

1  #include <sys/socket.h>
2  #include <netinet/in.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <stdio.h>
6
7  int main(void)
8  {
9      int server_fd, client_fd;
10     struct sockaddr_in addr = {
11         .sin_family      = AF_INET,
12         .sin_addr.s_addr = INADDR_ANY,
13         .sin_port        = htons(8080),
14     };
15     char buf[256];
16
17     /* socket() -> bind() -> listen() -> accept() -> read/write */
18     server_fd = socket(AF_INET, SOCK_STREAM, 0);
19
20     int opt = 1;
21     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
22     bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
23     listen(server_fd, 5);
24
25     printf("Server in ascolto su :8080\n");
26
27     while (1) {
28         client_fd = accept(server_fd, NULL, NULL);
29         ssize_t n = read(client_fd, buf, sizeof(buf) - 1);
30         if (n > 0) {
31             buf[n] = '\0';
32             printf("Ricevuto: %s\n", buf);
33             write(client_fd, "OK\n", 3);
34         }
35         close(client_fd);
36     }

```

37 }

7.3 Netfilter e nftables

Listing 7.3: Netfilter: hook points e nftables

```

1  # Hook points netfilter (dove iptables/nftables inseriscono regole):
2  # PREROUTING -> prima del routing (DNAT qui)
3  # INPUT      -> per pacchetti destinati a questo host
4  # FORWARD   -> per pacchetti in transito
5  # OUTPUT     -> generati da questo host
6  # POSTROUTING -> dopo il routing (SNAT/MASQUERADE qui)
7
8  # nftables: sintassi moderna
9  sudo nft list ruleset
10
11 # Crea firewall base con nftables
12 sudo nft -f - << 'EOF'
13 flush ruleset
14
15 table inet filter {
16     chain input {
17         type filter hook input priority 0; policy drop;
18
19         ct state established,related accept
20         iif lo accept
21         ip protocol icmp accept
22         tcp dport 22 accept      # SSH
23
24         # Log e drop il resto
25         log prefix "nft-drop: " drop
26     }
27
28     chain forward {
29         type filter hook forward priority 0; policy drop;
30     }
31
32     chain output {
33         type filter hook output priority 0; policy accept;
34     }
35 }
36
37 table ip nat {
38     chain postrouting {
39         type nat hook postrouting priority 100;
40         oifname "eth0" masquerade      # NAT/masquerade
41     }
42 }
43 EOF
44
45 # Salva regole permanenti
46 sudo nft list ruleset > /etc/nftables.conf
47
48 # iptables compatibilit  (usa nf_tables internamente)
49 sudo iptables -L -n -v --line-numbers
50 sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

```

7.4 XDP: eXpress Data Path

XDP permette di processare pacchetti a livello driver, prima che vengano allocati skb, con latenza minima.

Listing 7.4: Programma XDP minimale in BPF

```

1  /* xdp_drop.bpf.c - droppa tutti i pacchetti UDP */
2  #include <linux/bpf.h>
3  #include <linux/if_ether.h>
4  #include <linux/ip.h>
5  #include <linux/udp.h>
6  #include <bpf/bpf_helpers.h>
7
8  SEC("xdp")
9  int xdp_prog(struct xdp_md *ctx)
10 {
11     void *data_end = (void *) (long) ctx->data_end;
12     void *data      = (void *) (long) ctx->data;
13
14     struct ethhdr *eth = data;
15     if ((void *) (eth + 1) > data_end)
16         return XDP_PASS;
17
18     if (eth->h_proto != __constant_htons(ETH_P_IP))
19         return XDP_PASS;
20
21     struct iphdr *ip = (void *) (eth + 1);
22     if ((void *) (ip + 1) > data_end)
23         return XDP_PASS;
24
25     /* Droppa pacchetti UDP */
26     if (ip->protocol == IPPROTO_UDP)
27         return XDP_DROP;
28
29     return XDP_PASS; /* passa tutto il resto al kernel */
30 }
31
32 char LICENSE[] SEC("license") = "GPL";

```

Listing 7.5: Caricare e monitorare programmi XDP

```

1  # Compila programma XDP
2  clang -O2 -target bpf -c xdp_drop.bpf.c -o xdp_drop.bpf.o
3
4  # Carica su interfaccia (native mode = massima velocita')
5  sudo ip link set dev eth0 xdp obj xdp_drop.bpf.o sec xdp
6
7  # Verifica
8  ip link show eth0 | grep xdp
9
10 # Rimuovi
11 sudo ip link set dev eth0 xdp off
12

```

```
13 # Con iproute2 e bpftool
14 sudo bpftool prog list
15 sudo bpftool net list dev eth0
16
17 # Statistiche XDP
18 sudo bpftool prog show id 42 --json
```

7.5 Diagnostica di rete avanzata

Listing 7.6: Diagnostica rete a livello kernel

```
1 # Socket statistics (ss: successore di netstat)
2 ss -tuln # porte in ascolto
3 ss -t state established # connessioni TCP stabilite
4 ss -o state time-wait # TIME_WAIT
5 ss -i # info TCP interne (rtt, cwnd...)
6 ss -e # info estese (UID, inode)
7
8 # Routing avanzato
9 ip route show table all # tutte le tabelle di routing
10 ip route get 8.8.8.8 # percorso verso un IP
11 ip rule list # policy routing
12
13 # Namespace di rete
14 ip netns list # lista namespace
15 sudo ip netns add test_ns # crea namespace
16 sudo ip netns exec test_ns ip addr # comanda nel namespace
17
18 # Capturing (tcpdump a livello kernel)
19 sudo tcpdump -i eth0 -nn -s 0 port 80 -w /tmp/cap.pcap
20 sudo tcpdump -r /tmp/cap.pcap # leggi cattura
21
22 # Performance TCP
23 ss -ti | grep -E "rtt|cwnd|retrans"
24
25 # Coda TX/RX del driver NIC
26 ethtool -S eth0 | grep -E "rx|tx" | head -20
27 ethtool -g eth0 # dimensione ring buffer
```

Capitolo 8

Interrupt e Gestione Hardware

8.1 Tipi di interrupt

Tabella 8.1: Tipi di interrupt in Linux

Tipo	Sorgente	Caratteristiche
Hardware IRQ	Periferica (NIC, disco)	Asincrono, alta priorita', non puo' bloccare
Eccezione CPU	Istruzione illegale, PF	Sincrono, es: #GP (General Protection), #PF (Page Fault)
Softirq	Kernel software	Eseguito fuori da interrupt context, con interruzioni abilitate
Tasklet	Kernel software	Come softirq ma seriale per tasklet
Workqueue	Kernel thread	Puo' dormire, usa process context

Listing 8.1: Registrare un interrupt handler

```
1 #include <linux/interrupt.h>
2
3 #define MY_IRQ 18          /* numero IRQ (es. scheda PCI) */
4
5 /*
6  * L'handler viene chiamato in interrupt context:
7  * - NON puo' dormire (no mutex, no schedule())
8  * - NON puo' accedere a user space
9  * - Deve essere velocissimo (microsecondi)
10  * - Usa spin_lock_irqsave per proteggere dati condivisi
11  */
12 static irqreturn_t my_irq_handler(int irq, void *dev_id)
13 {
14     struct my_device *dev = dev_id;
15
16     /* Leggi registro status del device per confermare interrupt */
17     u32 status = readl(dev->base + STATUS_REG);
18     if (!(status & IRQ_FLAG))
19         return IRQ_NONE;    /* non era il nostro interrupt */
20
21     /* Azione minima: schedula il lavoro pesante nel bottom half */
22     tasklet_schedule(&dev->tasklet);
23
24     /* ACK interrupt all'hardware */
25     writel(IRQ_FLAG, dev->base + STATUS_REG);
26
27     return IRQ_HANDLED;
28 }
29
```

```

30  /* Registra l'handler all'init del modulo */
31  static int __init my_init(void)
32  {
33      int ret;
34      ret = request_irq(
35          MY_IRQ,
36          my_irq_handler,
37          IRQF_SHARED,      /* IRQ condivisibile con altri device */
38          "my_device",      /* nome visibile in /proc/interrupts */
39          my_dev            /* cookie per identificare il device */
40      );
41      if (ret) {
42          pr_err("Impossibile registrare IRQ %d: %d\n", MY_IRQ, ret);
43          return ret;
44      }
45      return 0;
46  }
47
48  static void __exit my_exit(void)
49  {
50      free_irq(MY_IRQ, my_dev);
51  }

```

8.2 Top half e bottom half

Il processing degli interrupt si divide in due fasi:

Listing 8.2: Tasklet come bottom half

```

1  #include <linux/interrupt.h>
2
3  static struct tasklet_struct my_tasklet;
4
5  /* Bottom half: eseguito con interrupt abilitati, puo' fare piu' lavoro */
6  static void my_tasklet_fn(unsigned long data)
7  {
8      struct my_device *dev = (struct my_device *)data;
9      /* elaborazione dati ricevuti, copia in buffer ring, ecc. */
10     pr_debug("Tasklet: elaborazione dati\n");
11 }
12
13 /* Init del tasklet */
14 tasklet_init(&my_tasklet, my_tasklet_fn, (unsigned long)my_dev);
15 /* Nel top half: tasklet_schedule(&my_tasklet); */
16 /* Cleanup: */ tasklet_kill(&my_tasklet);
17
18 /* WORKQUEUE: puo' dormire
19
20 */
21 #include <linux/workqueue.h>
22
23 static struct work_struct my_work;
24
25 static void my_work_fn(struct work_struct *work)
26 {
27     /* Qui possiamo: dormire, allocare con GFP_KERNEL, ecc. */

```

```

26     msleep(10);    /* attendi 10ms: IMPOSSIBILE in softirq/tasklet */
27     pr_info("Work completato\n");
28 }
29
30 INIT_WORK(&my_work, my_work_fn);
31 /* Nel top half: schedule_work(&my_work); */

```

Listing 8.3: Diagnostica interrupt

```

1  # Interrupt per CPU (colonne) per IRQ (righe)
2  cat /proc/interrupts
3  # CPU0      CPU1      CPU2      CPU3
4  # 0:   36   0   0   0   IO-APIC 2-edge   timer
5  # 1:    2   0   1   0   IO-APIC 1-edge   i8042
6  # 8:    0   0   0   1   IO-APIC 8-edge   rtc0
7  # 29: 12345 6789 ...      xhci_hcd
8
9  # Statistiche softirq
10 cat /proc/softirqs
11 # NET_TX NET_RX BLOCK TASKLET SCHED HRTIMER RCU ...
12
13 # IRQ affinity: quale CPU gestisce ogni IRQ
14 cat /proc/irq/29/smp_affinity      # bitmask hex
15 cat /proc/irq/29/smp_affinity_list # formato lista CPU
16
17 # Imposta IRQ affinity per bilanciamento
18 echo "2" | sudo tee /proc/irq/29/smp_affinity # solo CPU1
19
20 # irqbalance: daemon che bilancia automaticamente gli IRQ
21 sudo systemctl status irqbalance
22
23 # Latenza interrupt (con cyclicttest)
24 sudo cyclicttest -m -n -p99 -i200 -l100000 2>&1 | tail -5

```

8.3 Sincronizzazione in interrupt context

Listing 8.4: spin_lock_irqsave: protezione dati condivisi

```

1  #include <linux/spinlock.h>
2
3  static DEFINE_SPINLOCK(my_lock);
4  static struct list_head my_list = LIST_HEAD_INIT(my_list);
5
6  /* In interrupt handler (top half) */
7  static irqreturn_t my_handler(int irq, void *data)
8  {
9      unsigned long flags;
10
11      /*
12       * spin_lock_irqsave:
13       * 1. Disabilita interrupt locali (salva flags)
14       * 2. Acquisisce spinlock
15       * Necessario se il lock e' usato anche da codice
16       * NON-interrupt (es: process context)
17       */

```

```

18     spin_lock_irqsave(&my_lock, flags);
19
20     /* Sezione critica: accesso a dati condivisi */
21     list_add(&new_entry->list, &my_list);
22
23     spin_unlock_irqrestore(&my_lock, flags);
24
25     return IRQ_HANDLED;
26 }
27
28 /* In process context (es: read() del driver) */
29 static ssize_t my_read(struct file *f, char __user *buf,
30                       size_t count, loff_t *pos)
31 {
32     unsigned long flags;
33
34     /* spin_lock_irqsave anche qui: previene deadlock con handler */
35     spin_lock_irqsave(&my_lock, flags);
36     /* ... accedi alla lista ... */
37     spin_unlock_irqrestore(&my_lock, flags);
38
39     return count;
40 }

```

8.4 DMA: Direct Memory Access

Listing 8.5: Uso del DMA API nel kernel

```

1  #include <linux/dma-mapping.h>
2
3  /* Alloca buffer DMA coerente (CPU e device vedono stesso dato) */
4  void *cpu_addr;
5  dma_addr_t dma_addr;
6
7  cpu_addr = dma_alloc_coherent(
8      &pdev->dev,          /* device */
9      4096,               /* dimensione */
10     &dma_addr,           /* indirizzo fisico per il device */
11     GFP_KERNEL
12 );
13
14 /* Usa dma_addr per programmare il DMA controller del device */
15 writel((u32)dma_addr, dev->base + DMA_ADDR_REG);
16 writel(4096,          dev->base + DMA_SIZE_REG);
17 writel(DMA_START,     dev->base + DMA_CTRL_REG);
18
19 /* Libera */
20 dma_free_coherent(&pdev->dev, 4096, cpu_addr, dma_addr);
21
22 /* Per buffer streaming (piu' efficiente, non sempre coerente) */
23 dma_addr = dma_map_single(&pdev->dev, kbuf, size, DMA_TO_DEVICE);
24 /* ... trasferimento ... */
25 dma_unmap_single(&pdev->dev, dma_addr, size, DMA_TO_DEVICE);

```

Capitolo 9

Sincronizzazione e Locking

Il kernel Linux è altamente concorrente: più CPU eseguono codice kernel simultaneamente, gli interrupt possono arrivare in qualsiasi momento. La sincronizzazione corretta è critica.

9.1 Primitive di sincronizzazione

Tabella 9.1: Primitive di sincronizzazione nel kernel Linux

Primitiva	Può dormire?	Uso tipico
spinlock_t	No	Sezioni critiche brevi, interrupt context
mutex	Sì	Sezioni critiche lunghe, process context
rwlock_t	No	Multi reader / single writer, breve
rwsem	Sì	Multi reader / single writer, lungo
semaphore	Sì	Controllo accesso (raro nel kernel moderno)
atomic_t	No	Contatori atomici, flags
seqlock	No	Read-mostly, aggiornamenti rari
RCU	No (reader)	Lookup strutture dati read-heavy

9.2 Spinlock

Listing 9.1: Spinlock: uso corretto

```
1 #include <linux/spinlock.h>
2
3 /* Dichiarazione */
4 static DEFINE_SPINLOCK(my_lock);
5
6 /* --- VARIANTI PRINCIPALI --- */
7
8 /* 1. Process context, nessun interrupt che usa il lock */
9 spin_lock(&my_lock);
10    /* sezione critica */
11 spin_unlock(&my_lock);
12
13 /* 2. Process context, interrupt handler usa il lock */
14 unsigned long flags;
15 spin_lock_irqsave(&my_lock, flags);
16    /* sezione critica */
17 spin_unlock_irqrestore(&my_lock, flags);
18
19 /* 3. Interrupt handler (top half) */
20 spin_lock(&my_lock);          /* interrupt già disabilitati */
21    /* sezione critica */
```

```

22 spin_unlock(&my_lock);
23
24 /* 4. Bottom half (softirq/tasklet) - disabilita solo BH */
25 spin_lock_bh(&my_lock);
26     /* sezione critica */
27 spin_unlock_bh(&my_lock);
28
29 /*
30  * REGOLE FONDAMENTALI degli spinlock:
31  * - NON acquisire due spinlock in ordine inverso (deadlock)
32  * - NON chiamare funzioni che dormono (schedule, mutex_lock,
33  *   msleep, kmalloc con GFP_KERNEL...)
34  * - Tienili per il minimo tempo possibile
35  * - Usa lockdep per rilevare deadlock in sviluppo
36  */

```

9.3 Mutex

Listing 9.2: Mutex: uso in process context

```

1 #include <linux/mutex.h>
2
3 static DEFINE_MUTEX(my_mutex);
4
5 /* Acquisizione bloccante (puo' dormire: solo in process context) */
6 mutex_lock(&my_mutex);
7     /* sezione critica: puo' contenere operazioni che dormono */
8     msleep(10);                               /* OK con mutex */
9     copy_to_user(ubuf, kbuf, count);          /* OK con mutex */
10 mutex_unlock(&my_mutex);
11
12 /* Non-bloccante: restituisce 0 se non acquisisce */
13 if (mutex_trylock(&my_mutex)) {
14     /* sezione critica */
15     mutex_unlock(&my_mutex);
16 } else {
17     pr_info("mutex occupato, riprovo dopo\n");
18 }
19
20 /* Interrompibile da segnali (utile per evitare hang) */
21 if (mutex_lock_interruptible(&my_mutex)) {
22     return -ERESTARTSYS; /* segnale ricevuto */
23 }
24 /* ... */
25 mutex_unlock(&my_mutex);

```

9.4 RCU: Read-Copy-Update

RCU è il meccanismo di sincronizzazione più performante del kernel per strutture dati read-heavy. I lettori non acquisiscono nessun lock; gli scrittori creano una nuova copia e sostituiscono il puntatore atomicamente.

Listing 9.3: RCU: lettura e aggiornamento

```

1 #include <linux/rcupdate.h>
2 #include <linux/slab.h>
3
4 struct config_data {
5     int value;
6     struct rcu_head rcu; /* per rcu_call_free */
7 };
8
9 /* Puntatore globale protetto da RCU */
10 static struct config_data __rcu *global_config;
11
12 /*          LETTORE: nessun lock
13
14     */
15 static int read_config(void)
16 {
17     struct config_data *cfg;
18     int val;
19
20     rcu_read_lock();          /* inizio sezione RCU (non puo' dormire) */
21     cfg = rcu_dereference(global_config);
22     if (cfg)
23         val = cfg->value;      /* accesso sicuro */
24     rcu_read_unlock();        /* fine sezione RCU */
25
26     return val;
27 }
28
29 /*          SCRITTORE: crea nuova copia, poi aggiorna atomicamente */
30 static void update_config(int new_value)
31 {
32     struct config_data *new_cfg, *old_cfg;
33
34     new_cfg = kmalloc(sizeof(*new_cfg), GFP_KERNEL);
35     new_cfg->value = new_value;
36
37     /* Acquisisce lock solo tra scrittori (non blocca i lettori) */
38     spin_lock(&updater_lock);
39     old_cfg = rcu_dereference_protected(global_config,
40                                         lockdep_is_held(&updater_lock));
41
42     /* Aggiorna il puntatore: atomico, lettori vedono subito new_cfg */
43     rcu_assign_pointer(global_config, new_cfg);
44     spin_unlock(&updater_lock);
45
46     /* Attende che tutti i lettori con riferimento a old_cfg finiscano */
47     synchronize_rcu(); /* oppure: call_rcu(&old_cfg->rcu, kfree_rcu); */
48     kfree(old_cfg);
49 }

```

9.5 Operazioni atomiche

Listing 9.4: atomic_t e atomic64_t

```

1 #include <linux/atomic.h>

```

```

2
3  /* Dichiarazione e inizializzazione */
4  atomic_t ref_count = ATOMIC_INIT(0);
5  atomic64_t byte_count = ATOMIC64_INIT(0);
6
7  /* Operazioni atomiche principali */
8  atomic_set(&ref_count, 1);          /* imposta valore */
9  int val = atomic_read(&ref_count); /* leggi valore */
10 atomic_inc(&ref_count);              /* incrementa */
11 atomic_dec(&ref_count);              /* decrementa */
12 atomic_add(5, &ref_count);          /* aggiunge n */
13
14 /* Operazioni con test */
15 if (atomic_dec_and_test(&ref_count)) {
16     /* val == 0: ultimo riferimento, libera risorsa */
17     kfree(oggetto);
18 }
19
20 /* Increment e restituisce valore precedente */
21 int old = atomic_fetch_inc(&ref_count);
22
23 /* Compare-and-swap (CAS) */
24 int expected = 5;
25 atomic_cmpxchg(&ref_count, expected, 10);
26 /* Se ref_count == 5, lo imposta a 10; restituisce valore precedente */
27
28 /* Bit operations atomiche */
29 unsigned long flags = 0;
30 set_bit(0, &flags); /* imposta bit 0 */
31 clear_bit(0, &flags); /* azzera bit 0 */
32 test_bit(0, &flags); /* legge bit 0 */
33 test_and_set_bit(0, &flags); /* legge e imposta atomicamente */

```

9.6 Lockdep: rilevamento deadlock

Listing 9.5: Abilitare e usare lockdep

```

1  # Lockdep e' abilitato nel kernel di sviluppo
2  zcat /proc/config.gz | grep CONFIG_LOCKDEP
3  # CONFIG_LOCKDEP=y (solo in kernel di debug, non in produzione)
4
5  # Kernel Debian con lockdep debug
6  sudo apt install linux-image-6.1.0-* -686-pae-dbg
7
8  # Rapporti lockdep in dmesg
9  dmesg | grep -A 30 "deadlock\|DEADLOCK\|lockdep"
10
11 # Abilita prove di lock manuale (per sviluppo)
12 echo 1 | sudo tee /proc/sys/kernel/lock_stat
13
14 # Statistiche contention per lock
15 cat /proc/lock_stat | head -30
16
17 # Warning tipico lockdep:
18 # =====
19 # WARNING: possible circular locking dependency detected

```

```

20 # lock A -> lock B -> lock A (deadlock circle!)
21 # =====

```

9.7 seqlock e read-mostly patterns

Listing 9.6: seqlock per dati letti molto e scritti raramente

```

1 #include <linux/seqlock.h>
2
3 static DEFINE_SEQLOCK(my_seqlock);
4 static struct timeval my_time;
5
6 /* Scrittore: acquisisce lock in scrittura */
7 void update_time(struct timeval *new_time)
8 {
9     write_seqlock(&my_seqlock);
10    my_time = *new_time;
11    write_sequnlock(&my_seqlock);
12 }
13
14 /* Lettore: retry se c'e' stata una scrittura durante la lettura */
15 struct timeval read_time(void)
16 {
17     struct timeval t;
18     unsigned int seq;
19
20     do {
21         seq = read_seqbegin(&my_seqlock);
22         t = my_time;
23     } while (read_seqretry(&my_seqlock, seq));
24     /* retry se il sequence number e' cambiato (scrittore attivo) */
25
26     return t;
27 }
28 /*
29  * seqlock e' usato nel kernel per jiffies_64, timespec64,
30  * e altre strutture aggiornate frequentemente da un solo
31  * scrittore ma lette da moltissimi lettori.
32  */

```

Attenzione

Regola d'oro della sincronizzazione kernel: Scegliere la primitiva più leggera adeguata allo scenario:

1. Semplice flag/contatore → `atomic_t`
2. Dati read-heavy → RCU o seqlock
3. Sezione critica breve, interrupt context → spinlock
4. Sezione critica lunga, process context → mutex
5. Multi-reader, long → rwsem

Capitolo 10

eBPF: Programmabilità del Kernel

eBPF (extended Berkeley Packet Filter) permette di eseguire programmi sandboxed nel kernel senza modificarlo e senza scrivere moduli. È la tecnologia più rivoluzionaria del kernel Linux degli ultimi anni.

10.1 Architettura eBPF

Tabella 10.1: Componenti del sottosistema eBPF

Componente	Funzione
Verifier	Analisi statica del programma BPF prima del caricamento: assenza di loop infiniti, accessi memoria validi, terminazione garantita
JIT Compiler	Compila bytecode BPF in istruzioni native x86_64/arm64 per prestazioni massime
Mappe BPF	Strutture dati condivise tra kernel e user space (hash, array, ringbuf, LRU...)
Helper functions	Funzioni kernel chiamabili dal programma BPF (bpf_map_lookup, bpf_ktime_get_ns...)
Hook points	Punti di aggancio: kprobe, tracepoint, XDP, socket, cgroup, LSM, sched...

10.2 Primo programma eBPF

Listing 10.1: Tracing di sys_execve con eBPF (BCC Python)

```
1 #!/usr/bin/env python3
2 """
3 execve_tracer.py -- traccia ogni exec() nel sistema
4 Richiede: sudo apt install bpftrace python3-bcc (Debian)
5             sudo pacman -S bcc (Arch)
6 """
7 from bcc import BPF
8
9 # Programma eBPF in C embedded in Python
10 prog = r"""
11 #include <uapi/linux/ptrace.h>
12 #include <linux/sched.h>
13
14 /* Struttura evento passata allo userspace via perf buffer */
15 struct event_t {
16     u32 pid;
17     u32 ppid;
18     char comm[TASK_COMM_LEN]; /* nome processo corrente */
19 }
```

```

19     char filename[256];           /* file eseguito */
20 };
21
22 BPF_PERF_OUTPUT(events); /* canale kernel -> userspace */
23
24 /* Aggancio: sys_execve e' chiamata ad ogni exec() */
25 TRACEPOINT_PROBE(syscalls, sys_enter_execve) {
26     struct event_t evt = {};
27     struct task_struct *task;
28
29     evt.pid = bpf_get_current_pid_tgid() >> 32;
30     evt.ppid = bpf_get_current_pid_tgid() & 0xffffffff;
31     bpf_get_current_comm(&evt.comm, sizeof(evt.comm));
32     bpf_probe_read_user_str(evt.filename, sizeof(evt.filename),
33                             (void *)args->filename);
34
35     events.perf_submit(args, &evt, sizeof(evt));
36     return 0;
37 }
38 """
39
40 b = BPF(text=prog)
41
42 def print_event(cpu, data, size):
43     evt = b["events"].event(data)
44     print(f"PID={evt.pid:5d} COMM={evt.comm.decode():16s} "
45           f"EXEC={evt.filename.decode()}")
46
47 b["events"].open_perf_buffer(print_event)
48 print("Tracciando exec()... Ctrl+C per uscire")
49 while True:
50     b.perf_buffer_poll()

```

Listing 10.2: Installare e usare strumenti eBPf

```

1 #           Installazione
2
3 # Debian/Kali
4 sudo apt install bpftrace bpfcc-tools linux-headers-$(uname -r)
5
6 # Arch Linux
7 sudo pacman -S bpf bcc bpftrace
8
9 #           bpftrace: linguaggio alto livello
10
11 # Traccia ogni open() nel sistema
12 sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat {
13     printf("%s -> %s\n", comm, str(args->filename));
14 }'
15
16 # Conta syscall per processo
17 sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter {
18     @[comm] = count();
19 } interval:s:10 { print(@); clear(@); }'

```

```

19 # Latenza operazioni disco
20 sudo bpftrace -e 'kprobe:blk_account_io_start {
21     @start[arg0] = nsecs;
22 } kprobe:blk_account_io_done /@start[arg0]/ {
23     @latency = hist(nsecs - @start[arg0]);
24     delete(@start[arg0]);
25 }'
26
27 #             bcc tools (script pronti)

28 # Latenza open() per processo
29 sudo opensnoop-bpfcc
30
31 # Top syscall
32 sudo syscount-bpfcc
33
34 # Cache hit rate filesystem
35 sudo cachestat-bpfcc 1
36
37 # Flame graph CPU (richiede FlameGraph)
38 sudo profile-bpfcc -F 99 30 | stackcollapse-bpf.pl | \
39     flamegraph.pl > cpu.svg
40
41 # Latenza rete TCP
42 sudo tcplife-bpfcc
43
44 # File aperti con stack trace
45 sudo opensnoop-bpfcc -x # solo errori

```

10.3 Mappe eBPf

Listing 10.3: Tipi di mappe BPF e loro uso

```

1  /* La mappa BPF e' la struttura dati condivisa kernel<->userspace */
2
3  /* Hash map: chiave -> valore, O(1) medio */
4  struct {
5      __uint(type, BPF_MAP_TYPE_HASH);
6      __uint(max_entries, 10240);
7      __type(key, u32); /* PID */
8      __type(value, u64); /* contatore */
9  } pid_count SEC(".maps");
10
11 /* Array: indice -> valore, accesso O(1) */
12 struct {
13     __uint(type, BPF_MAP_TYPE_ARRAY);
14     __uint(max_entries, 256);
15     __type(key, u32);
16     __type(value, u64);
17 } syscall_count SEC(".maps");
18
19 /* Ring buffer: kernel -> userspace, zero-copy */
20 struct {
21     __uint(type, BPF_MAP_TYPE_RINGBUF);
22     __uint(max_entries, 256 * 1024); /* 256 KB */

```

```

23 } events SEC(".maps");
24
25 /* Nel programma BPF: */
26 u32 key = bpf_get_current_pid_tgid() >> 32;
27 u64 *count = bpf_map_lookup_elem(&pid_count, &key);
28 if (count)
29     __sync_fetch_and_add(count, 1);
30 else {
31     u64 init = 1;
32     bpf_map_update_elem(&pid_count, &key, &init, BPF_NOEXIST);
33 }
34
35 /* Ring buffer: emit evento zero-copy */
36 struct my_event *evt = bpf_ringbuf_reserve(&events,
37     sizeof(*evt), 0);
38 if (evt) {
39     evt->pid = key;
40     bpf_ringbuf_submit(evt, 0);
41 }

```

10.4 bpftool: ispezione del sottosistema eBPF

Listing 10.4: bpftool: gestione e diagnostica programmi BPF

```

1  # Lista programmi BPF caricati nel kernel
2  sudo bpftool prog list
3  # 42: kprobe name my_kprobe tag abc123...
4  #   loaded_at 2026-06-13T10:00:00+0200 uid 0
5  #   xlated 240B jited 256B memlock 4096B
6
7  # Dump bytecode BPF di un programma
8  sudo bpftool prog dump xlated id 42
9
10 # Dump JIT compilato (istruzioni x86)
11 sudo bpftool prog dump jited id 42
12
13 # Lista mappe BPF
14 sudo bpftool map list
15 sudo bpftool map dump id 5          # contenuto mappa
16
17 # Programmi BPF agganciati alle interfacce
18 sudo bpftool net list dev eth0
19
20 # Feature supportate dal kernel
21 sudo bpftool feature probe
22
23 # Struttura BTF (type info per CO-RE)
24 sudo bpftool btf list
25 sudo bpftool btf dump id 1 format c | head -30
26
27 # Statistiche per programma (richiede kernel 5.1+)
28 sudo bpftool prog stats id 42
29 # run_cnt: 12345, run_time_ns: 6789012

```

10.5 CO-RE: portabilità tra kernel

Listing 10.5: CO-RE: Compile Once, Run Everywhere

```
1  /*
2   * CO-RE (Compile Once, Run Everywhere) permette di scrivere
3   * programmi eBPf che funzionano su kernel diversi senza
4   * ricompilazione. Usa BTF (BPF Type Format) per adattare
5   * gli accessi ai campi delle strutture kernel a runtime.
6   */
7
8  /* vmlinux.h: generato da BTF del kernel target */
9  /* sudo bpftool btf dump file /sys/kernel/btf/vmlinux
10     format c > vmlinux.h */
11
12  #include "vmlinux.h"
13  #include <bpf/bpf_helpers.h>
14  #include <bpf/bpf_core_read.h>
15
16  SEC("kprobe/do_sys_open")
17  int trace_open(struct pt_regs *ctx)
18  {
19      struct task_struct *task = (struct task_struct *)bpf_get_current_task()
20          ;
21
22      /*
23       * BPF_CORE_READ: accede al campo in modo portabile.
24       * Il verifier/loader adatta l'offset al kernel corrente
25       * leggendo le info BTF.
26       */
27      u32 pid = BPF_CORE_READ(task, pid);
28      u64 start_time = BPF_CORE_READ(task, start_time);
29
30      bpf_printk("open: pid=%d start=%llu\n", pid, start_time);
31      return 0;
32  }
33
34  char LICENSE[] SEC("license") = "GPL";
```

Capitolo 11

Namespaces, Cgroups e Container

I namespace e i cgroup sono i building block fondamentali dei container Linux (Docker, Podman, LXC). Forniscono isolamento (namespace) e controllo delle risorse (cgroups).

11.1 Namespace Linux

Tabella 11.1: I sette tipi di namespace Linux

Tipo	Flag clone()	Isola
mnt	CLONE_NEWNS	Punti di mount (/proc/mounts)
pid	CLONE_NEWPID	APID dei processi (PID 1 nel namespace)
net	CLONE_NEWNET	Interfacce, routing, iptables, socket
ipc	CLONE_NEWIPC	Code messaggi, semafori, shared memory
uts	CLONE_NEWUTS	Hostname e domainname
user	CLONE_NEWUSER	UID/GID mapping (root nel NS → utente reale)
cgroup	CLONE_NEWCGROUP	Vista gerarchica cgroups

Listing 11.1: Creare e usare namespace con unshare e ip netns

```
1 # Visualizza namespace del processo corrente
2 ls -la /proc/self/ns/
3 # lrwxrwxrwx net -> net:[4026531992]
4 # lrwxrwxrwx pid -> pid:[4026531836]
5
6 # Crea nuovo namespace di rete (isolato)
7 sudo ip netns add mio_ns
8 sudo ip netns list
9 sudo ip netns exec mio_ns ip addr          # nessuna interfaccia
10
11 # Crea coppia veth per connettere i namespace
12 sudo ip link add veth0 type veth peer name veth1
13 sudo ip link set veth1 netns mio_ns
14 sudo ip addr add 10.0.0.1/24 dev veth0
15 sudo ip netns exec mio_ns ip addr add 10.0.0.2/24 dev veth1
16 sudo ip link set veth0 up
17 sudo ip netns exec mio_ns ip link set veth1 up
18 ping 10.0.0.2      # comunicazione tra i due namespace
19
20 # unshare: crea namespace senza fork
21 sudo unshare --pid --mount-proc --fork bash
22 # Ora sei in un namespace PID isolato: ps mostra solo i tuoi proc
23
24 # Namespace UTS: hostname isolato
25 sudo unshare --uts bash
```

```

26 hostname mio-container      # non visibile fuori
27 hostname                    # mio-container
28 exit
29 hostname                    # host originale inalterato
30
31 # Namespace user: diventa root senza essere root
32 unshare --user --map-root-user bash
33 id      # uid=0(root) gid=0(root) <-- root nel namespace
34 cat /proc/self/uid_map      # 0 1000 1 (NS_root -> host_uid 1000)

```

Listing 11.2: Creazione namespace via clone() in C

```

1 #define _GNU_SOURCE
2 #include <sched.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <sys/wait.h>
7
8 #define STACK_SIZE (1024 * 1024)
9
10 static int child_fn(void *arg)
11 {
12     printf("Figlio PID (nel namespace): %d\n", getpid()); /* sempre 1 */
13     printf("Figlio hostname: ");
14     sethostname("container", 9);
15     execlp("bash", "bash", NULL);
16     return 0;
17 }
18
19 int main(void)
20 {
21     char *stack = malloc(STACK_SIZE);
22     char *stack_top = stack + STACK_SIZE;
23
24     pid_t pid = clone(child_fn,
25                      stack_top,
26                      /* Crea namespace PID, UTS e mount */
27                      CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS |
28                      SIGCHLD,
29                      NULL);
30
31     printf("Padre: PID figlio (host) = %d\n", pid);
32     waitpid(pid, NULL, 0);
33     free(stack);
34     return 0;
35 }

```

11.2 Cgroups v2

Listing 11.3: Gestione cgroups v2

```

1 # Verifica che cgroups v2 sia montato
2 mount | grep cgroup2
3 # cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec)

```

```

4
5 # Struttura gerarchica
6 ls /sys/fs/cgroup/
7 # cpu.max memory.max io.max pids.max ... system.slice/
8
9 # Crea un cgroup per limitare risorse
10 sudo mkdir /sys/fs/cgroup/mio_gruppo
11
12 # Limita CPU al 50% (500ms ogni 1000ms)
13 echo "500000 1000000" | \
14   sudo tee /sys/fs/cgroup/mio_gruppo/cpu.max
15
16 # Limita memoria a 256 MB
17 echo "268435456" | \
18   sudo tee /sys/fs/cgroup/mio_gruppo/memory.max
19
20 # Limita numero di processi
21 echo "10" | sudo tee /sys/fs/cgroup/mio_gruppo/pids.max
22
23 # Aggiungi processo al cgroup
24 echo $$ | sudo tee /sys/fs/cgroup/mio_gruppo/cgroup.procs
25
26 # Limita I/O disco (in byte/s per dispositivo)
27 # Trova il major:minor del disco
28 ls -la /dev/sda # 8:0
29 echo "8:0 rbps=10485760 wbps=10485760" | \
30   sudo tee /sys/fs/cgroup/mio_gruppo/io.max
31
32 # Verifica uso risorse
33 cat /sys/fs/cgroup/mio_gruppo/cpu.stat
34 cat /sys/fs/cgroup/mio_gruppo/memory.current
35 cat /sys/fs/cgroup/mio_gruppo/io.stat
36
37 # Interfaccia systemd per cgroups
38 systemctl set-property --runtime sshd.service \
39   MemoryMax=100M CPUQuota=20%
40 systemctl status sshd.service | grep -A5 "CGroup:"

```

11.3 Container: come funziona Docker internamente

Listing 11.4: Anatomia di un container Docker

```

1 # Avvia container e ispeziona i namespace
2 docker run -d --name test nginx
3
4 # PID del processo nel container (visto dall'host)
5 docker inspect test --format '{{.State.Pid}}'
6 # 12345
7
8 # Namespace del container (diversi dall'host)
9 ls -la /proc/12345/ns/
10 # net -> net:[4026532XXX] (diverso da host!)
11 # pid -> pid:[4026532YYY]
12 # mnt -> mnt:[4026532ZZZ]
13
14 # Entra nel namespace del container

```

```

15 sudo nsenter -t 12345 --net --pid bash
16 # Ora sei nel network namespace del container
17 ip addr # vedi solo le interfacce del container
18
19 # Cgroup del container
20 cat /proc/12345/cgroup
21 # 0::/system.slice/docker-ABC123.scope
22
23 # Limiti del container
24 cat /sys/fs/cgroup/system.slice/docker-ABC123.scope/memory.max
25
26 # Overlayfs: il filesystem del container
27
28 # Docker usa overlayfs per i layer dell'immagine
29 cat /proc/mounts | grep overlay
30 # overlay /var/lib/docker/overlay2/... overlay
31 # lowerdir=/var/lib/docker/...
32 # upperdir=/var/lib/docker/.../diff (layer scrivibile)
33 # workdir=/var/lib/docker/.../work

```

11.4 Seccomp: filtro system call

Listing 11.5: Seccomp BPF: limitare le syscall di un processo

```

1 #include <linux/seccomp.h>
2 #include <linux/filter.h>
3 #include <sys/prctl.h>
4 #include <sys/syscall.h>
5
6 /* Installa un filtro seccomp: permette solo read, write, exit */
7 static int install_seccomp_filter(void)
8 {
9     struct sock_filter filter[] = {
10         /* Carica il numero di syscall */
11         BPF_STMT(BPF_LD|BPF_W|BPF_ABS,
12                 offsetof(struct seccomp_data, nr)),
13         /* Permetti read, write, exit, exit_group */
14         BPF_JUMP(BPF_JMP|BPF_JEQ|BPF_K, __NR_read, 3, 0),
15         BPF_JUMP(BPF_JMP|BPF_JEQ|BPF_K, __NR_write, 2, 0),
16         BPF_JUMP(BPF_JMP|BPF_JEQ|BPF_K, __NR_exit, 1, 0),
17         BPF_JUMP(BPF_JMP|BPF_JEQ|BPF_K, __NR_exit_group, 0, 1),
18         /* Permetti */
19         BPF_STMT(BPF_RET|BPF_K, SECCOMP_RET_ALLOW),
20         /* Blocca: invia SIGSYS */
21         BPF_STMT(BPF_RET|BPF_K, SECCOMP_RET_KILL),
22     };
23
24     struct sock_fprog prog = {
25         .len = sizeof(filter) / sizeof(filter[0]),
26         .filter = filter,
27     };
28
29     /* Richiede PR_SET_NO_NEW_PRIVS prima */
30     if (prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0))
31         return -1;
32

```

```
33     if (syscall(SYS_seccomp, SECCOMP_SET_MODE_FILTER, 0, &prog))
34         return -1;
35
36     return 0;
37 }
```

Capitolo 12

Sicurezza del Kernel: LSM e Hardening

12.1 Linux Security Modules (LSM)

Il framework LSM inserisce hook in punti critici del kernel (operazioni su file, socket, processi, ecc.) e permette a moduli di sicurezza di applicare policy aggiuntive. LSM è lo strato sottostante a SELinux, AppArmor, Smack, TOMOYO e altri.

Listing 12.1: Hook LSM: esempio su operazioni file

```
1  /* include/linux/lsm_hooks.h (semplificato) */
2
3  /* L'LSM registra callback per ogni operazione sensibile */
4  struct security_hook_list {
5      struct hlist_node    list;
6      struct hlist_head    *head;
7      union security_list_options hook;
8      const char            *lsm;
9  };
10
11  /*
12   * Esempi di hook disponibili:
13   *
14   * inode_permission: prima di ogni accesso a file
15   *   int (*inode_permission)(struct inode *inode, int mask);
16   *
17   * file_open: apertura di un file
18   *   int (*file_open)(struct file *file);
19   *
20   * socket_connect: prima di connect()
21   *   int (*socket_connect)(struct socket *sock,
22   *                           struct sockaddr *address, int addrlen);
23   *
24   * task_kill: prima di inviare un segnale
25   *   int (*task_kill)(struct task_struct *p, struct kernel_siginfo *info,
26   *                     int sig, const struct cred *cred);
27   *
28   * bprm_check_security: prima di exec()
29   *   int (*bprm_check_security)(struct linux_binprm *bprm);
30   */
31
32  /* Chiamata nel VFS (fs/namei.c) */
33  int inode_permission(struct inode *inode, int mask)
34  {
35      int ret;
36
37      /* Prima verifica permessi DAC (Unix tradizionale) */
38      ret = do_inode_permission(inode, mask);
39      if (ret)
```

```

40     return ret;
41
42     /* Poi chiama gli hook LSM (MAC aggiuntivo) */
43     return security_inode_permission(inode, mask);
44 }

```

12.2 Capabilities Linux

Le capability sostituiscono il modello binario root/non-root con un insieme di privilegi granulari.

Tabella 12.1: Capability Linux principali

Capability	Permette
CAP_CHOWN	Cambiare proprietario di qualsiasi file
CAP_DAC_OVERRIDE	Ignorare i permessi DAC (lettura/scrittura/exec)
CAP_NET_ADMIN	Configurazione rete, iptables, interfacce
CAP_NET_RAW	Socket RAW (ping, tcpdump)
CAP_SYS_ADMIN	Tutte le operazioni amministrative (quasi tutto)
CAP_SYS_PTRACE	ptrace su processi altrui
CAP_SYS_MODULE	Caricare/rimuovere moduli kernel
CAP_SYS_RAWIO	Accesso I/O raw (ioperm, iopl)
CAP_SETUID	Cambiare UID arbitrariamente
CAP_BPF	Caricare programmi eBPF

Listing 12.2: Gestione capability

```

1  # Capability di un processo
2  cat /proc/self/status | grep Cap
3  # CapInh: 0000000000000000 (ereditabili)
4  # CapPrm: 0000000000000000 (permesse)
5  # CapEff: 0000000000000000 (effettive)
6  # CapBnd: 000001ffffffff (bounding set)
7
8  # Decodifica capability mask
9  capsh --decode=000001ffffffff
10 # 0x000001ffffffff=cap_chown,cap_dac_override,...
11
12 # Verifica capability di un processo
13 getpcaps $$
14 getpcaps $(pgrep sshd | head -1)
15
16 # Dai capability specifiche a un eseguibile (no SUID)
17 sudo setcap cap_net_raw,cap_net_admin+ep /usr/bin/tcpdump
18 # Rimuovi
19 sudo setcap -r /usr/bin/tcpdump
20 # Verifica
21 getcap /usr/bin/tcpdump
22
23 # Esegui processo con capability ridotte
24 sudo capsh --drop=cap_sys_admin -- -c "bash"
25 # In questo bash: CAP_SYS_ADMIN non disponibile

```

12.3 Hardening del kernel

Listing 12.3: Parametri sysctl di hardening

```

1  #                               File /etc/sysctl.d/99-hardening.conf
2  cat << 'EOF' | sudo tee /etc/sysctl.d/99-hardening.conf
3
4  # Proteggi ptrace: solo parent puo' ptrace i figli
5  kernel.yama.ptrace_scope = 1
6  # 0 = tutti, 1 = solo parent, 2 = solo CAP_SYS_PTRACE, 3 = off
7
8  # ASLR massima
9  kernel.randomize_va_space = 2
10
11 # Proteggi symlink e hardlink in /tmp
12 fs.protected_symlinks = 1
13 fs.protected_hardlinks = 1
14
15 # Disabilita core dump da SUID
16 fs.suid_dumpable = 0
17
18 # Disabilita magic SysRq in produzione
19 kernel.sysrq = 0
20
21 # Restrizioni su moduli (no loading dopo init)
22 # kernel.modules_disabled = 1 # ATTENZIONE: irreversibile!
23
24 # Rete: prevenzione spoofing
25 net.ipv4.conf.all.rp_filter = 1
26 net.ipv4.conf.all.accept_source_route = 0
27 net.ipv4.conf.all.accept_redirects = 0
28 net.ipv4.conf.all.log_martians = 1
29 net.ipv4.tcp_syncookies = 1
30
31 # Prevenzione IP forwarding (se non router)
32 net.ipv4.ip_forward = 0
33 net.ipv6.conf.all.forwarding = 0
34
35 EOF
36
37 sudo sysctl --system # applica tutti i file in sysctl.d/

```

Listing 12.4: Secure Boot e verifica integrita' del kernel

```

1  #                               Verificare Secure Boot
2
3  mokutil --sb-state
4  # SecureBoot enabled oppure SecureBoot disabled
5
6  # Firma moduli kernel (richiesto con Secure Boot attivo)
7  # Genera coppia di chiavi MOK
8  openssl req -new -x509 -newkey rsa:2048 \
9    -keyout MOK.priv -out MOK.der \
10   -days 3650 -subj "/CN=Mio Modulo Kernel/"

```

```

11 # Firma il modulo
12 sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file \
13     sha256 MOK.priv MOK.der mio_modulo.ko
14
15 # Importa la chiave nel MOK (Machine Owner Key) di shim
16 sudo mokutil --import MOK.der
17 # Al prossimo avvio: conferma nel MOK manager UEFI
18
19 #             dm-verity: verifica integrita' filesystem
20
21 # Crea hash tree per un volume
22 sudo veritysetup format /dev/sdb1 /dev/sdb2
23
24 # Root hash: abc123... (da salvare e verificare al boot)
25
26 # Monta con verifica
27 sudo veritysetup open /dev/sdb1 verified /dev/sdb2 abc123...
28 sudo mount /dev/mapper/verified /mnt/verified -o ro
29
30 #             Kernel lockdown mode
31
32 # Restringe ulteriormente l'accesso al kernel anche da root
33 cat /sys/kernel/security/lockdown
34 # [none] integrity confidentiality
35
36 # integrity: blocca accesso raw a /dev/mem, /dev/kmem, BPF JIT...
37 echo integrity | sudo tee /sys/kernel/security/lockdown

```

12.4 AppArmor: profili per il kernel

Listing 12.5: Creare profili AppArmor personalizzati

```

1 # Genera profilo da zero per un'applicazione
2 sudo aa-genprof /usr/local/bin/mio_server
3 # Lancia il programma, AppArmor logga tutto
4 # Premi S per scansionare i log e costruire il profilo
5
6 # Profilo manuale (/etc/apparmor.d/usr.local.bin.mio_server)
7 cat << 'EOF' | sudo tee /etc/apparmor.d/mio_server
8 #include <tunables/global>
9
10 /usr/local/bin/mio_server {
11     #include <abstractions/base>
12     #include <abstractions/nameservice>
13
14     # File permessi
15     /etc/mio_server/config r,      # solo lettura config
16     /var/log/mio_server/ rw,      # lettura/scrittura log
17     /var/log/mio_server/** rw,
18
19     # Rete
20     network tcp,                  # TCP permesso
21     network udp,                  # UDP permesso
22
23     # Eseguibili
24     /usr/bin/logger ix,           # puo' eseguire logger

```

```
25
26     # Nega tutto il resto (implicito)
27 }
28 EOF
29
30 # Carica il profilo
31 sudo apparmor_parser -r /etc/apparmor.d/mio_server
32
33 # Modalita' enforce vs complain
34 sudo aa-enforce /usr/local/bin/mio_server
35 sudo aa-complain /usr/local/bin/mio_server
36
37 # Verifica violazioni
38 sudo journalctl | grep apparmor | tail -10
```

Appendice A

Strumenti di Analisi e Debug

Tabella A.1: Strumenti fondamentali per l'analisi del kernel

Tool	Categoria	Funzione
dmesg	Log kernel	Ring buffer messaggi kernel
journalctl -k	Log kernel	Log kernel via systemd
perf	Profiling	CPU, eventi hardware, syscall, tracing
ftrace	Tracing	Function tracer del kernel
bpftrace	eBPF tracing	Linguaggio alto livello per eBPF
strace	Syscall	Traccia le syscall di un processo
ltrace	Librerie	Traccia chiamate a librerie condivise
gdb + kgdb	Debugger	Debug kernel a basso livello
crash	Post-mortem	Analisi vmcore dump
valgrind	Memoria	Memory leak, uso errato memoria
kasan	Memoria	Kernel Address SANitizer
kcsan	Concorrenza	Kernel Concurrency SANitizer
syzkaller	Fuzzing	Fuzzer automatico per syscall
stress-ng	Stress test	Stress test componenti kernel

Listing A.1: perf: profiling avanzato

```
1 # Installa
2 sudo apt install linux-perf      # Debian/Kali
3 sudo pacman -S perf             # Arch
4
5 # Profile CPU (campionamento a 99 Hz per 10 secondi)
6 sudo perf record -F 99 -a -g -- sleep 10
7 sudo perf report
8
9 # Conta eventi hardware durante un comando
10 sudo perf stat -e cache-misses,cache-references,\
11 instructions,cycles ls /etc
12
13 # Traccia syscall in tempo reale
14 sudo perf trace -a --duration 5
15
16 # Genera flame graph
17 sudo perf record -F 99 -a -g -- sleep 30
18 sudo perf script | stackcollapse-perf.pl | flamegraph.pl > cpu.svg
```

Listing A.2: ftrace: il tracer integrato nel kernel

```
1 # Accedi al filesystem di tracing
2 cd /sys/kernel/debug/tracing
3
4 # Lista tracers disponibili
```

```

5 cat available_tracers
6 # blk function function_graph irqsoff preemptoff wakeup nop
7
8 # Attiva function tracer
9 echo function | sudo tee current_tracer
10 echo 1 | sudo tee tracing_on
11 cat trace | head -30
12 echo 0 | sudo tee tracing_on
13
14 # Traccia solo una funzione specifica
15 echo "tcp_sendmsg" | sudo tee set_ftrace_filter
16 echo function | sudo tee current_tracer
17 echo 1 | sudo tee tracing_on
18 # ... genera traffico TCP ...
19 cat trace
20
21 # Function graph: visualizza call stack con tempi
22 echo function_graph | sudo tee current_tracer
23 echo 1 | sudo tee tracing_on
24 sleep 0.1
25 echo 0 | sudo tee tracing_on
26 cat trace | head -50
27
28 # Trace eventi specifici (tracepoint)
29 echo 1 | sudo tee events/sched/sched_switch/enable
30 cat trace | grep sched_switch | head -10

```

Listing A.3: kgdb: debug del kernel in esecuzione

```

1 # Abilita kgdb nel kernel (parametri boot)
2 # In /etc/default/grub:
3 # GRUB_CMDLINE_LINUX="kgdboc=ttyS0,115200 kgdbwait"
4
5 # Connetti da macchina remota
6 gdb vmlinux
7 (gdb) target remote /dev/ttyS0
8 (gdb) continue
9
10 # Con QEMU (sviluppo kernel)
11 qemu-system-x86_64 \
12   -kernel arch/x86/boot/bzImage \
13   -s -S \      # -s=gdbserver :1234, -S=pausa all'avvio
14   -append "console=ttyS0 nokaslr"
15 # In un'altra shell:
16 gdb vmlinux
17 (gdb) target remote :1234
18 (gdb) break sys_execve
19 (gdb) continue

```

Appendice B

Fonti e Riferimenti

B.1 Documentazione ufficiale

The Linux Kernel documentation <https://docs.kernel.org>

Documentazione ufficiale del kernel: API, sottosistemi, guide allo sviluppo.

Linux Kernel Internals <https://kernel-internals.org>

Guide tecniche dettagliate su CFS/EEVDF, memoria, BPF.

Bootlin Elixir Cross Referencer <https://elixir.bootlin.com/linux/latest/source>

Navigazione interattiva del sorgente kernel con cross-reference delle strutture.

LWN.net — Linux Weekly News <https://lwn.net>

Articoli tecnici approfonditi sulle novità del kernel.

kernel.org git <https://git.kernel.org>

Repository git ufficiale; log dei commit con spiegazioni.

B.2 Libri di riferimento

Linux Kernel Programming (2024) Kaiwan N. Billimoria. Packt Publishing, 2024. Basato su kernel 6.1 LTS. ISBN: 978-1803232225.

Linux Kernel Programming: Device Drivers (2025) Thierry Gayet. Packt Publishing, 2025. Focus su driver char, block, USB, PCI.

Understanding the Linux Kernel (3rd ed.) Bovet & Cesati. O'Reilly, 2005. Datato ma ancora utile per i concetti fondamentali.

Linux Device Drivers (3rd ed.) Corbet, Rubini, Kroah-Hartmann. O'Reilly, 2005. Disponibile gratuitamente: <https://lwn.net/Kernel/LDD3/>

BPF Performance Tools Brendan Gregg. Addison-Wesley, 2019. Il riferimento per eBPF, bpftrace e analisi performance.

B.3 Sorgenti online

Arch Wiki — Kernel <https://wiki.archlinux.org/title/Kernel>

Brendan Gregg's Linux Performance <https://www.brendangregg.com/linuxperf.html>

kernel-internals.org — CFS <https://kernel-internals.org/sched/cfs/>

Linux Foundation — LFD420 <https://training.linuxfoundation.org/training/linux-kernel-int>

Glossario

Termini tecnici usati nel documento con **significato**, **dove viene usato** e **funzioni principali**.

ASLR — Address Space Layout Randomization *Significato:* Tecnica di sicurezza che randomizza la posizione in memoria di codice, stack e heap a ogni esecuzione. *Dove:* Implementato nel kernel (`mm/mmap.c`); configurabile via `/proc/sys/kernel/randomize_va_space`. *Funzioni:* Rende impraticabili attacchi buffer-overflow che dipendono da indirizzi fissi. Valore 2 = protezione massima (randomizza anche stack e vDSO).

BPF / eBPF — extended Berkeley Packet Filter *Significato:* Framework per eseguire programmi sandboxed nel kernel senza modificarlo, verificati staticamente dal kernel verifier. *Dove:* `kernel/bpf/`; hook in kprobe, tracepoint, XDP, cgroup, LSM, sched_ext. *Funzioni:* Tracing, profiling, networking (XDP, tc), sicurezza (LSM BPF), scheduling personalizzato (sched_ext). Il verifier garantisce terminazione e assenza di accessi non autorizzati.

BTF — BPF Type Format *Significato:* Formato compatto di informazioni di tipo (simile a DWARF ma ottimizzato) embedded nel kernel e nei programmi BPF. *Dove:* `/sys/kernel/btf/vmlinux`; usato dal loader libbpf per CO-RE. *Funzioni:* Permette a programmi BPF compilati una volta di adattarsi a kernel diversi a runtime (CO-RE: Compile Once, Run Everywhere), leggendo gli offset dei campi delle strutture dal BTF del kernel corrente.

CFS — Completely Fair Scheduler *Significato:* Scheduler predefinito per processi normali (SCHED_NORMAL) basato sul concetto di *vruntime*. *Dove:* `kernel/sched/fair.c`; attivo su tutti i sistemi Linux moderni. *Funzioni:* Garantisce equità tra task assegnando CPU al task con *vruntime* minore. Dal kernel 6.6 usa EEVDF per la selezione. Organizza i task in un albero rosso-nero.

cgroup — Control Group *Significato:* Meccanismo kernel per raggruppare processi e applicare limiti di risorse in modo gerarchico. *Dove:* `kernel/cgroup/`; montato in `/sys/fs/cgroup` (v2); usato da systemd, Docker, Kubernetes. *Funzioni:* Limita e monitora CPU, memoria, I/O e numero di processi per gruppo. cgroups v2 unifica la gerarchia in un albero singolo con semantica migliorata rispetto a v1.

CO-RE — Compile Once, Run Everywhere *Significato:* Tecnica per scrivere programmi eBPF portabili tra versioni diverse del kernel. *Dove:* Libreria libbpf; file `vmlinux.h` generato da BTF. *Funzioni:* Usa `BPF_CORE_READ()` per accedere ai campi delle strutture kernel tramite BTF, adattando gli offset automaticamente al kernel target a runtime.

DMA — Direct Memory Access *Significato:* Meccanismo hardware che permette ai dispositivi di trasferire dati in RAM senza coinvolgere la CPU. *Dove:* `include/linux/dma-mapping.h`;

usato da praticamente ogni driver di dispositivo moderno. *Funzioni:* Riduce il carico CPU durante I/O intensivi (disco, rete, GPU). Il kernel espone un'API unificata (`dma_alloc_coherent`, `dma_map_single`) che gestisce la coerenza della cache tra CPU e device.

EEVDF — Earliest Eligible Virtual Deadline First *Significato:* Algoritmo di selezione del task introdotto nel kernel 6.6 in sostituzione del classico approccio leftmost del CFS. *Dove:* `kernel/sched/fair.c`, funzione `pick_eevdf()`. *Funzioni:* Assegna a ogni task una *virtual deadline* e seleziona il task *eligible* (`vruntime > min_vruntime`) con deadline minore. Migliora la prevedibilità della latenza rispetto al puro CFS.

GFP — Get Free Pages flags *Significato:* Flag passati alle funzioni di allocazione memoria del kernel per descrivere il contesto e i vincoli. *Dove:* `include/linux/gfp.h`; usati da `kmalloc`, `alloc_pages`, `kmem_cache_alloc`. *Funzioni:* `GFP_KERNEL` (può dormire, memoria normale), `GFP_ATOMIC` (non può dormire, interrupt context), `GFP_DMA` (memoria DMA), `__GFP_ZERO` (azzerà pagina).

IOMMU — Input-Output Memory Management Unit *Significato:* Unità hardware che gestisce la traduzione degli indirizzi per i dispositivi DMA. *Dove:* `drivers/iommu/`; Intel VT-d, AMD-Vi; usato da VFIO per passthrough GPU. *Funzioni:* Isola i dispositivi tra loro impedendo a un device compromesso di accedere a tutta la RAM. Necessario per la virtualizzazione sicura dei dispositivi (VFIO, SR-IOV).

IRQ — Interrupt ReQuest *Significato:* Segnale asincrono inviato da una periferica alla CPU per richiedere attenzione immediata. *Dove:* `kernel/irq/`; configurazione in `/proc/interrupts` e `/proc/irq/`. *Funzioni:* Permette alla CPU di rispondere a eventi hardware (pacchetto di rete arrivato, settore disco letto) senza polling. Il codice dell'handler (top half) deve essere brevissimo; il lavoro pesante viene delegato al bottom half (softirq, tasklet, workqueue).

kprobes *Significato:* Meccanismo di tracing dinamico che permette di agganciare codice a qualsiasi istruzione del kernel. *Dove:* `kernel/kprobes.c`; usato da perf, SystemTap, bpftrace, BCC. *Funzioni:* Inserisce un breakpoint (int3 su x86) nella funzione target. Quando raggiunto, chiama l'handler registrato. Usato per tracing, profiling e debugging senza ricompilare il kernel.

LRU — Least Recently Used *Significato:* Algoritmo di sostituzione pagine usato dal gestore della memoria per scegliere quale pagina spostare su swap o rimuovere dalla page cache. *Dove:* `mm/vmscan.c`; liste `active` e `inactive` in `struct pglist_data`. *Funzioni:* Le pagine meno usate recentemente vengono spostate dalla lista `active` a quella `inactive`, e poi eventualmente liberate o swappate. Bilanciamento controllato da `vm.swappiness`.

LSM — Linux Security Module *Significato:* Framework che inserisce hook di sicurezza in punti critici del kernel, permettendo a moduli di sicurezza di applicare policy MAC. *Dove:* `security/`; hook definiti in `include/linux/lsm_hooks.h`. *Funzioni:* Base di SELinux, AppArmor, Smack. Il VFS, il network stack e altri sottosistemi chiamano gli hook LSM prima di eseguire operazioni sensibili. Il risultato può bloccare l'operazione anche se i permessi DAC sono soddisfatti.

NAPI — New API *Significato:* Meccanismo di interrupt mitigation per driver di rete ad alto traffico. *Dove:* `net/core/dev.c`; implementato da ogni driver NIC moderno. *Funzioni:* Invece di un interrupt per ogni pacchetto, dopo il primo interrupt il driver disabilita gli IRQ e processa pacchetti in polling (`napi_poll`). Riduce drasticamente il numero di interrupt a bassa latenza.

OOM Killer — Out Of Memory Killer *Significato:* Componente del kernel che sceglie e termina un processo quando il sistema esaurisce la memoria. *Dove:* `mm/oom_kill.c`; attivato da `__alloc_pages_slowpath`. *Funzioni:* Calcola un `oom_score` (0–1000) per ogni processo basandosi su memoria usata, durata e altri fattori. Termina il processo con score più alto. Configurabile via `/proc/PID/oom_score_adj`.

page cache *Significato:* Cache in RAM del contenuto di file e dispositivi a blocchi, gestita dal kernel per accelerare I/O. *Dove:* `mm/filemap.c`; struttura `address_space` nell'inode. *Funzioni:* Ogni lettura da disco popola la page cache; letture successive sono servite dalla RAM. Le scritture vanno in cache (dirty pages) e vengono scritte su disco in modo asincrono dal thread `kworker/flush`.

RCU — Read-Copy-Update *Significato:* Meccanismo di sincronizzazione kernel ottimizzato per strutture dati lette frequentemente e modificate raramente. *Dove:* `kernel/rcu/`; usato pesantemente in network stack, VFS, scheduler. *Funzioni:* I lettori non acquisiscono nessun lock (overhead zero); gli scrittori creano una nuova copia del dato, aggiornano il puntatore atomicamente, e aspettano che tutti i lettori esistenti terminino prima di liberare la vecchia copia.

sched_ext *Significato:* Framework introdotto in kernel 6.12 che permette di implementare scheduler personalizzati tramite eBPF. *Dove:* `kernel/sched/ext.c`; hook `enqueue/dispatch/select_cpu`. *Funzioni:* Consente a ricercatori e aziende (Google, Meta) di sperimentare nuovi algoritmi di scheduling senza patch kernel. Dal 6.19: fault recovery automatico con fallback a CFS in millisecondi, riduzione latenza fino al 15% in scenari ad alta contesa.

sk_buff *Significato:* Struttura dati centrale del network stack Linux che rappresenta un pacchetto di rete. *Dove:* `include/linux/skbuff.h`; allocato in `alloc_skb()`, liberato in `kfree_skb()`. *Funzioni:* Contiene i dati del pacchetto e tutti i metadati (puntatori agli header di layer 2/3/4, device, socket, timestamp, conntrack). I puntatori `head/data/tail/end` permettono di aggiungere/rimuovere header senza copiare i dati.

SLUB Allocator *Significato:* Allocatore di oggetti a dimensione fissa del kernel, successore di SLAB. *Dove:* `mm/slub.c`; usato da `kmalloc`, `kmem_cache_alloc`. *Funzioni:* Mantiene cache per-CPU di oggetti pre-allocati e inizializzati (es: `task_struct`, `inode`, `dentry`). Riduce la frammentazione e migliora le performance rispetto all'allocazione pagine per oggetti piccoli. Visibile in `/proc/slabinfo` e `slabtop`.

softirq *Significato:* Interrupt software eseguito fuori dal contesto di interrupt hardware, con priorità elevata. *Dove:* `kernel/softirq.c`; tipi: HI, TIMER, NET_TX, NET_RX, BLOCK, SCHED, RCU. *Funzioni:* Gestisce il lavoro differito dall'interrupt handler (top half). Può

essere preempted da interrupt hardware ma ha priorità su processi normali. Le statistiche sono visibili in `/proc/softirqs`.

spinlock *Significato:* Primitiva di sincronizzazione che “gira” (busy-wait) finché il lock non è disponibile. *Dove:* `include/linux/spinlock.h`; usato ovunque nel kernel dove è necessaria sincronizzazione veloce. *Funzioni:* Adatto per sezioni critiche brevi in interrupt context (non può dormire). Le varianti `irqsave/irqrestore` disabilitano gli interrupt per prevenire deadlock con gli handler. Deve essere tenuto per il minor tempo possibile.

task_struct *Significato:* La struttura dati più importante del kernel: rappresenta ogni processo e thread in esecuzione. *Dove:* `include/linux/sched.h`; circa 700 campi nel kernel 6.x. *Funzioni:* Contiene tutto ciò che il kernel sa di un task: PID, TGID, credenziali, stato, priorità, puntatori a `mm_struct` (memoria), `files_struct` (file descriptor), `signal_struct` (segnali), `nsproxy` (namespace) e `cgroups`.

VFS — Virtual Filesystem Switch *Significato:* Strato di astrazione che presenta un’interfaccia unificata a tutti i filesystem (`open`, `read`, `write`...) indipendentemente dall’implementazione sottostante. *Dove:* `fs/`; strutture in `include/linux/fs.h`. *Funzioni:* Traduce le chiamate POSIX nelle operazioni specifiche del filesystem (`ext4`, `xfs`, `btrfs`, `tmpfs`...) tramite puntatori a funzione in `file_operations`, `inode_operations` e `super_operations`.

vruntime — Virtual Runtime *Significato:* Misura del tempo CPU virtuale consumato da un task nel CFS scheduler. *Dove:* Campo `se.vruntime` in `task_struct`; aggiornato in `update_curr()` ad ogni tick. *Funzioni:* Il vruntime cresce proporzionalmente al tempo reale, pesato inversamente per la priorità (nice). Task con nice basso (alta priorità) accumulano vruntime più lentamente, ottenendo più CPU.

workqueue *Significato:* Meccanismo per eseguire lavoro differito in un contesto di processo kernel (kthread), a differenza di tasklet e softirq. *Dove:* `kernel/workqueue.c`; API: `INIT_WORK`, `schedule_work`, `queue_work`. *Funzioni:* A differenza di tasklet/softirq, le funzioni nelle workqueue *possono dormire* (mutex, allocazione, I/O). Ideali per operazioni lunghe demandate da interrupt handler. Il sistema di default (`system_wq`) usa kthread condivisi.

XDP — eXpress Data Path *Significato:* Framework per processare pacchetti di rete a livello driver NIC, prima dell’allocazione di `sk_buff`. *Dove:* `net/core/xdp.c`; hook nel driver NIC; programmi BPF agganciati con `ip link set xdp`. *Funzioni:* Permette di accettare (`XDP_PASS`), scartare (`XDP_DROP`), ridirigere (`XDP_REDIRECT`) o ritrasmettere (`XDP_TX`) pacchetti a velocità linea (milioni di pacchetti/secondo) con latenza nanosecondo.